

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
CORSO DI LAUREA IN INGEGNERIA INFORMATICA
CORSO DI INGEGNERIA DEL SOFTWARE
PROF. A.R. FASOLINO - A.A. 2025-26...



“SISTEMA SOFTWARE PER GESTIONE DI ALLENAMENTI SPORTIVI PER ATLETI”

INDICE

1 SPECIFICHE INFORMALI

2. ANALISI E SPECIFICA DEI REQUISITI

- 2.1 ANALISI NOMI-VERBI
- 2.2 REVISIONE DEI REQUISITI
- 2.3 GLOSSARIO DEI TERMINI
- 2.4 CLASSIFICAZIONE DEI REQUISITI
 - 2.4.1 REQUISITI FUNZIONALI
 - 2.4.2 REQUISITI SUI DATI
 - 2.4.3 REQUISITI NON FUNZIONALI DI QUALITÀ
 - 2.4.4 VINCOLI / ALTRI REQUISITI
- 2.5 MODELLAZIONE DEI CASI D'USO
 - 2.5.1 ATTORI E CASI D'USO
 - 2.5.2 DIAGRAMMA DEI CASI D'USO
 - 2.5.3 SCENARI DEI CASI D'USO
 - UC1: REGISTRAZIONE
 - UC2: ACCESSO
 - UC6: ASSOCIAZIONE DIRETTA
 - UC7: ASSOCIAZIONE CODICE UNIVOCO
 - UC13: REGISTRAZIONE RISULTATI
 - UC9: CREAZIONE SESSIONE
 - UC17: MONITORAGGIO DASHBOARD
- 2.6 DIAGRAMMA DELLE CLASSI CONCETTUALE
 - 2.6.1 RESPONSABILITÀ (GRASP)
- 2.7 DIAGRAMMI DI SEQUENZA CONCETTUALI
- 2.8 DIAGRAMMA DELLE CLASSI RAFFINATO

3. PIANO DI TEST FUNZIONALE (CATEGORY PARTITION)

- 3.1 SUITE DI TEST: REGISTRAZIONE
- 3.2 SUITE DI TEST: ACCESSO
- 3.3 SUITE DI TEST: ASSOCIAZIONE DIRETTA
- 3.4 SUITE DI TEST: ASSOCIAZIONE CODICE UNIVOCO
- 3.5 SUITE DI TEST: REGISTRAZIONE RISULTATI
- 3.6 SUITE DI TEST: MONITORAGGIO DASHBOARD
- 3.7 SUITE DI TEST: CREAZIONE SESSIONE

4. PROGETTAZIONE

- 4.1 DIAGRAMMA DELLE CLASSI DI PROGETTAZIONE
 - 4.1.1 TRADUZIONE CLASSI ED ASSOCIAZIONI
 - 4.1.2 SCELTE ARCHITETTURALI E PATTERN BCED
 - 4.1.2.1 PACKAGE BOUNDARY
 - 4.1.2.2 PACKAGE CONTROLLER
 - 4.1.2.3 PACKAGE ENTITY (PATTERN FACADE)
 - 4.1.2.4 PACKAGE DATABASE
- 4.2 DIAGRAMMI DI SEQUENZA DI PROGETTO
 - 4.2.1 REGISTRAZIONE
 - 4.2.2 ACCESSO
 - 4.2.3 ASSOCIAZIONE DIRETTA
 - 4.2.4 ASSOCIAZIONE CODICE UNIVOCO
 - 4.2.5 REGISTRAZIONE RISULTATI
 - 4.2.6 MONITORAGGIO DASHBOARD
 - 4.2.7 CREAZIONE SESSIONE
- 4.3 DIAGRAMMA DELLE CLASSI DI PROGETTAZIONE DI DETTAGLIO

5. IMPLEMENTAZIONE

- 5.1 PACKAGE DATABASE (CLASSI JPAUTIL, GESTOREPERSISTENZA)
- 5.2 PACKAGE ENTITY (CLASSI DEL DOMINIO E FACADE APPSPORT)
- 5.3 PACKAGE CONTROLLER (CLASSI GESTOREUTENTI, GESTORESESSIONI, GESTOREDASHBOARD)

- **5.4 PACKAGE BOUNDARY (INTERFACCE GRAFICHE SWING E GESTIONE EVENTI)**
- **5.5 DIAGRAMMA DI DEPLOYMENT E SPECIFICHE DI INSTALLAZIONE**
- 6. TESTING**
 - **6.1 TEST STRUTTURALE (WHITE-BOX)**
 - **6.1.1 COMPLESSITÀ CICLOMATICA E CONTROL FLOW GRAPH**
 - **6.1.1.1 METODO: CREA NUOVA SESSIONE – APPSPORT**
 - **6.1.1.2 METODO: ASSOCIA CON CODICE – APPSPORT**
 - **6.2 JUNIT – TEST DI UNITÀ**
 - **6.3 TEST FUNZIONALE (RESOCONTO ESECUZIONE DELLA TEST SUITE)**
 - **6.3.1 CLASSI DI TEST DI UNITÀ ED ESITO DEI CASI D'USO**

1. Specifiche informali

Si desidera sviluppare un sistema software per la gestione di allenamenti sportivi personalizzati destinati ad atleti seguiti da uno o più allenatori. Il sistema dovrà consentire agli allenatori di pianificare sessioni di allenamento, monitorare i risultati ottenuti e consultare lo storico delle attività svolte, mentre gli atleti potranno visualizzare i propri allenamenti, registrare le prestazioni e seguire il piano assegnato.

Descrizione del Sistema:

Il sistema consente la registrazione di utenti tramite credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo, scegliendo tra atleta o allenatore, oltre a fornire nome, cognome, email e, opzionalmente, una disciplina sportiva prevalente.

Ogni allenatore ha la possibilità di creare e gestire il profilo di uno o più atleti associati. Per ciascun atleta è possibile definire informazioni di base, quali disciplina praticata, livello di esperienza ed eventuali obiettivi sportivi. L'associazione tra atleta e allenatore può avvenire direttamente da parte dell'allenatore oppure, per semplicità, tramite un codice univoco fornito all'atleta, che potrà così collegarsi al proprio allenatore.

L'allenatore, mediante apposita interfaccia grafica, può creare sessioni di allenamento personalizzate. Ogni sessione è definita da un titolo, una descrizione, una data di svolgimento ed eventualmente una durata prevista. Una sessione può includere uno o più esercizi, ciascuno caratterizzato da un nome, una descrizione, un numero di ripetizioni o una durata prevista, a seconda della tipologia di attività.

Ogni atleta autenticato dispone di una propria interfaccia nella quale può visualizzare l'elenco delle sessioni assegnate, ordinate cronologicamente. Selezionando una sessione, l'atleta può consultare il dettaglio degli esercizi previsti e, al termine dell'allenamento, registrare i risultati effettivamente ottenuti. Per ciascun esercizio, il sistema deve consentire di inserire almeno un valore osservato, come ad esempio il numero effettivo di ripetizioni, il tempo impiegato oppure una nota testuale relativa alla prestazione.

Una volta che l'atleta ha completato l'inserimento dei dati relativi a una sessione, la sessione può essere marcata come "completata". In alternativa, può rimanere nello stato "assegnata" finché non viene registrato almeno un risultato. Per semplicità, si può supporre che una sessione possa assumere uno dei seguenti stati: "assegnata", "in corso" e "completata". L'allenatore può consultare in ogni momento lo storico degli allenamenti svolti da ciascun atleta, visualizzando sia il piano originariamente assegnato sia i risultati effettivamente registrati. Deve inoltre essere possibile accedere al dettaglio delle singole sessioni e monitorare l'andamento delle prestazioni nel tempo.

Ogni atleta ha accesso a una sezione del proprio profilo personale in cui può consultare il numero totale di allenamenti completati, l'elenco delle sessioni svolte e quelle ancora da eseguire. L'allenatore, attraverso un'interfaccia dedicata, può monitorare l'andamento complessivo dei propri atleti, visualizzando il numero di sessioni assegnate, il numero di sessioni completate e, opzionalmente, indicatori sintetici relativi alla continuità degli allenamenti.



Il sistema può includere una funzione di ricerca e filtraggio delle sessioni per data, disciplina o stato. Deve inoltre essere possibile visualizzare il profilo sportivo dell'atleta, comprensivo almeno delle informazioni anagrafiche essenziali, della disciplina praticata e degli obiettivi assegnati.

Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche che avvisi l'atleta dell'assegnazione di un nuovo allenamento e l'allenatore del completamento di una sessione. Il sistema dovrà inoltre garantire la protezione dei dati personali, la sicurezza delle informazioni salvate e una corretta gestione dei permessi tra atleti e allenatori.



2. Analisi e specifica dei requisiti

2.1 Analisi nomi-verbi

Il sistema consente la registrazione di utenti tramite credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo, scegliendo tra atleta o allenatore, oltre a fornire nome, cognome, email e, opzionalmente, una disciplina sportiva prevalente.

Ogni allenatore ha la possibilità di creare e gestire il profilo di uno o più atleti associati. Per ciascun atleta è possibile definire informazioni di base, quali disciplina praticata, livello di esperienza ed eventuali obiettivi sportivi. L'associazione tra atleta e allenatore può avvenire direttamente da parte dell'allenatore oppure, per semplicità, tramite un codice univoco fornito all'atleta, che potrà così collegarsi al proprio allenatore.

L'allenatore, mediante apposita interfaccia grafica, può creare sessioni di allenamento personalizzate. Ogni sessione è definita da un titolo, una descrizione, una data di svolgimento ed eventualmente una durata prevista. Una sessione può includere uno o più esercizi, ciascuno caratterizzato da un nome, una descrizione, un numero di ripetizioni o una durata prevista, a seconda della tipologia di attività.

Ogni atleta autenticato dispone di una propria interfaccia nella quale può visualizzare l'elenco delle sessioni assegnate, ordinate cronologicamente. Selezionando una sessione, l'atleta può consultare il dettaglio degli esercizi previsti e, al termine dell'allenamento, registrare i risultati effettivamente ottenuti. Per ciascun esercizio, il sistema deve consentire di inserire almeno un valore osservato, come ad esempio il numero effettivo di ripetizioni, il tempo impiegato oppure una nota testuale relativa alla prestazione.

Una volta che l'atleta ha completato l'inserimento dei dati relativi a una sessione, la sessione può essere marcata come "completata". In alternativa, può rimanere nello stato "assegnata" finché non viene registrato almeno un risultato. Per semplicità, si può supporre che una sessione possa assumere uno dei seguenti stati: "assegnata", "in corso" e "completata".

L'allenatore può consultare in ogni momento lo storico degli allenamenti svolti da ciascun atleta, visualizzando sia il piano originariamente assegnato sia i risultati effettivamente registrati. Deve inoltre essere possibile accedere al dettaglio delle singole sessioni e monitorare l'andamento delle prestazioni nel tempo.

Ogni atleta ha accesso a una sezione del proprio profilo personale in cui può consultare il numero totale di allenamenti completati, l'elenco delle sessioni svolte e quelle ancora da eseguire. L'allenatore, attraverso un'interfaccia dedicata, può monitorare l'andamento complessivo dei propri atleti, visualizzando il numero di sessioni assegnate, il numero di sessioni completate e, opzionalmente, indicatori sintetici relativi alla continuità degli allenamenti.

Il sistema può includere una funzione di ricerca e filtraggio delle sessioni per data, disciplina o stato. Deve inoltre essere possibile visualizzare il profilo sportivo dell'atleta, comprensivo almeno delle informazioni anagrafiche essenziali, della disciplina praticata e degli obiettivi assegnati.

Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche che avvisi l'atleta dell'assegnazione di un nuovo allenamento e l'allenatore del completamento di una sessione. Il sistema dovrà inoltre garantire la protezione dei dati personali, la sicurezza delle informazioni salvate e una corretta gestione dei permessi tra atleti e allenatori.



LEGENDA:

Classe

Attributo

Funzionalità

Attore

Classe-Attore

2.2 Revisione dei requisiti

1. Il sistema deve offrire all'Utente una funzionalità per registrarsi tramite credenziali personali (nome, cognome ed e-mail).
2. l'utente, In fase di registrazione, deve poter specificare il proprio ruolo (atleta o allenatore).
3. Il sistema deve offrire all'Utente una funzionalità per accedere (login).
4. Il sistema deve memorizzare, oltre ai dati anagrafici, l'eventuale disciplina sportiva prevalente.
5. Il sistema deve offrire all'Allenatore una funzionalità per creare e gestire il profilo di uno o più atleti associati.
6. Il sistema deve permettere all' Allenatore di associare se stesso a uno o più atleti (metodo diretto)
7. Il sistema deve fornire all'allenatore un codice univoco per poter associare se stesso al profilo di un atleta (in alternativa al metodo diretto)
8. Di ogni Atleta si vuole memorizzare disciplina praticata, livello di esperienza ed eventuali obiettivi sportivi.
9. Il sistema deve offrire all'Allenatore una funzionalità per creare sessioni di allenamento personalizzate, specificando titolo, descrizione, data e durata prevista.
10. Il sistema memorizza per ogni sessione l'elenco degli esercizi, ciascuno caratterizzato da nome, descrizione e parametri previsti (ripetizioni o durata).
11. Il sistema deve offrire all'Atleta una funzionalità per visualizzare l'elenco delle proprie sessioni assegnate, ordinate cronologicamente.
12. Il sistema deve offrire all'Atleta una funzionalità per registrare i risultati effettivamente ottenuti (ripetizioni, tempi o note testuali) per ogni esercizio svolto.
13. Il sistema deve gestire lo stato di ogni sessione, permettendo il passaggio tra gli stati: "assegnata", "in corso" e "completata".
14. Il sistema deve offrire all'Allenatore una funzionalità per visualizzare l'elenco cronologico (lo storico) di tutte le sessioni di allenamento passate per ogni singolo atleta a lui associato.
15. Il sistema deve mostrare all'Allenatore, per ogni sessione nello storico, sia i parametri dell'allenamento originariamente assegnato (es. ripetizioni/tempi previsti) sia i risultati reali registrati dall'atleta.
16. Il sistema deve permettere all'Allenatore di selezionare una singola sessione dallo storico per aprirne il dettaglio completo (es. singoli esercizi, note inserite, parametri specifici).
17. Il sistema deve fornire all'Allenatore una funzionalità per analizzare l'evoluzione e la progressione delle prestazioni dell'atleta nel corso del tempo.
18. Il sistema deve offrire all'Atleta una sezione profilo per consultare il totale degli allenamenti completati e l'elenco delle sessioni future.
19. Il sistema deve offrire all'Allenatore un'interfaccia dedicata per monitorare il numero di sessioni assegnate/completate e indicatori sulla continuità degli allenamenti dei propri atleti.



20. Il sistema deve permettere all'allenatore di selezionare un singolo atleta dall'interfaccia dedicata (punto 18) per visualizzarne il profilo sportivo, comprensivo delle informazioni anagrafiche essenziali, della disciplina praticata e degli obiettivi assegnati.
21. Il sistema deve offrire funzionalità di ricerca e filtraggio delle sessioni per data, disciplina o stato.
22. Il sistema deve inviare una notifica all'Atleta all'assegnazione di un nuovo allenamento e all'Allenatore al completamento di una sessione.
23. Il sistema deve garantire la protezione dei dati personali e il controllo dei permessi di accesso in base al ruolo.
24. Il sistema deve essere intuitivo e responsivo per l'utilizzo
25. Il sistema deve essere multi-piattaforma, accessibile via web, da desktop che da mobile.

2.3 Glossario dei termini

Termine	Descrizione	Sinonimi
Utente	Qualsiasi individuo che interagisce con il sistema previa registrazione tramite credenziali personali. Può assumere il ruolo di Allenatore o Atleta.	
Allenatore	Utente con permessi di livello superiore che ha la capacità di creare, gestire e monitorare uno o più Atleti, assegnare piani di allenamento e visionare i risultati.	
Atleta	Utente che esegue i piani di allenamento. Ha accesso alla visualizzazione delle sessioni assegnate, alla registrazione dei risultati e al monitoraggio delle proprie statistiche.	
Profilo	L'insieme delle informazioni sportive di un atleta	Profilo Atleta
Codice Univoco	Singola attività fisica inclusa in una Sessione, caratterizzata da un nome, una descrizione e parametri oggettivi (es. numero di ripetizioni previste o durata prevista).	
Sessione	Un'unità di lavoro programmata dall'Allenatore in una specifica data. È composta da uno o più Esercizi ed è identificata da un titolo e una descrizione.	Allenamento, Attività, Piano
Esercizio		
Valore Osservato	Il dato effettivo registrato dall'Atleta al termine dell'esecuzione di un Esercizio. Può essere un valore numerico (ripetizioni fatte, tempo impiegato) o una nota testuale qualitativa.	Risultati effettivamente ottenuti, Prestazione

2.4 Classificazione dei requisiti

2.4.1 Requisiti funzionali

ID	Requisito	Origine (n. frase dei
----	-----------	-----------------------



		requisiti revisionati)
RF01	Il sistema deve offrire all'Utente una funzionalità per registrarsi tramite credenziali personali.	1
RF02	Il sistema deve offrire all'Utente una funzionalità per accedere (login).	3
RF03	Il sistema deve offrire all'Allenatore una funzionalità per creare e gestire il profilo degli atleti associati.	5
RF04	Il sistema deve permettere all'Allenatore di associare al proprio account uno o più atleti tramite metodo diretto.	6
RF05	Il sistema deve fornire all'allenatore un codice univoco per associare se stesso al profilo di un atleta	7
RF06	Il sistema deve offrire all'Allenatore una funzionalità per creare sessioni di allenamento personalizzate.	9
RF07	Il sistema deve offrire all'Atleta una funzionalità per visualizzare l'elenco cronologico delle proprie sessioni assegnate.	11
RF08	Il sistema deve offrire all'Atleta una funzionalità per registrare i risultati effettivamente ottenuti per ogni esercizio.	12
RF09	Il sistema deve offrire all'Allenatore una funzionalità per visualizzare lo storico cronologico delle sessioni di ogni atleta associato.	14
RF10	Il sistema deve mostrare all'Allenatore il confronto tra i parametri pianificati e i risultati reali registrati dall'atleta.	15
RF11	Il sistema deve permettere all'Allenatore di accedere al dettaglio completo di una singola sessione dello storico.	16
RF12	Il sistema deve fornire all'Allenatore una funzionalità per analizzare l'evoluzione delle prestazioni dell'atleta nel tempo.	17
RF13	Il sistema deve offrire all'Atleta una sezione profilo per consultare il totale degli allenamenti completati e le sessioni future.	18
RF14	Il sistema deve offrire all'Allenatore una dashboard per monitorare aggregati e indicatori di continuità degli atleti.	19
RF15	Il sistema deve permettere all'Allenatore di visualizzare il profilo sportivo completo di un atleta selezionato dalla dashboard.	20
RF16	Il sistema deve offrire funzionalità di ricerca e filtraggio delle sessioni per data, disciplina o stato.	21
RF17	Il sistema deve inviare notifiche automatiche per l'assegnazione (all'atleta) o il completamento (all'allenatore) di un allenamento.	22

2.4.2 Requisiti sui dati

ID	Requisito	Origine (n. frase dei requisiti revisionati)
----	-----------	--



RD01	Di ogni Utente si memorizzano: nome, cognome, e-mail e disciplina sportiva prevalente	4
RD02	Di ogni Profilo Atleta si memorizzano: disciplina, livello di esperienza e obiettivi sportivi.	5
RD03	Di ogni Sessione si memorizzano: titolo, descrizione, data e durata prevista.	
RD04	Di ogni Esercizio si memorizzano: nome, descrizione e parametri ripetizioni, durata	
RD05	Di ogni Prestazione si memorizza un valore osservato: che rappresenta: ripetizioni effettive, tempi o note testuali	

2.4.3 Requisiti Non Funzionali di Qualità (si può fare riferimento alle caratteristiche dello Standard ISO 25010)

ID	Categoria	Descrizione	Metrica	Valore atteso	Verifica
RQ-QUAL-01	Sicurezza (Riservatezza)	Il sistema deve garantire la protezione dei dati e il controllo dei permessi basato sul ruolo.	Tentativi di accesso a dati altrui non autorizzati andati a buon fine.	0 (Zero)	Test di vulnerabilità
RQ-QUAL-02	Usabilità (Apprendibilità)	L'interfaccia deve essere intuitiva per consentire all'atleta di registrare i risultati senza assistenza.	Tempo medio per la prima registrazione di un allenamento.	≤ 3 minuti	Test di usabilità con un campione di utenti reali.
RQ-QUAL-03	Compatibilità (Interoperabilità)	Il sistema deve poter essere eseguito su differenti dispositivi			Test di compatibilità automatizzati.
RQ-QUAL-04	Usabilità (Operatività)	L'interfaccia deve essere responsiva e adattarsi correttamente ai diversi dispositivi.	Tempo di riposizionamento del layout al cambio di risoluzione/orientamento.	≤ 500 ms.	Test di performance tramite strumenti specifici



RQ-QUAL-05	Sicurezza (Integrità)	Garanzia della sicurezza delle informazioni salvate nel sistema.	Integrità dei dati dopo un ripristino da backup.	100% coerenza dati	Test di Disaster Recovery e verifica Checksum dei database.
RQ-QUAL-06	Efficienza Prestazionale	Il sistema deve essere responsivo in termini di velocità di esecuzione delle operazioni	Tempo di risposta del sistema a seguito di un'interazione utente	≤ 1 secondo (90% dei casi)	Test dei tempi di risposta

2.4.4 Vincoli / Altri requisiti

ID	Requisito	Origine (n. frase dei requisiti revisionati)
V01	Obbligo di scelta del ruolo (atleta/allenatore) esclusivamente in fase di registrazione.	2
V02	L'associazione atleta-allenatore è permessa solo tramite metodo diretto o codice univoco.	6,7
V03	Gli stati della sessione sono limitati a: "assegnata", "in corso" e "completata".	13
V04	Un allenatore può visualizzare solo i dati degli atleti a lui formalmente associati.	23
V05	L'allenatore può creare profili univocamente ad atleti ad esso associati	3



2.5 Modellazione dei casi d'uso

2.5.1 Attori e casi d'uso

Attori Primari:

- Utente
- Allenatore
- Atleta

Attori Secondari:

- Servizio di messaggistica

Casi d'uso:

- **UC1:** Registrazione
- **UC2:** Accesso
- **UC3:** CreazioneProfiloAtleta
- **UC4:** EliminazioneProfiloAtleta
- **UC5:** ModificaProfiloAtleta
- **UC6:** AssociazioneDiretta
- **UC7:** AssociazioneCodiceUnivoco
- **UC8:** VisualizzazioneProfiloAtleta
- **UC9:** CreazioneSessione
- **UC10:** ModificaSessione
- **UC11:** EliminazioneSessione
- **UC12:** VisualizzazioneCronologiaSessioni
- **UC13:** RegistrazioneRisultati
- **UC14:** ConsultazioneStoricoAtleti
- **UC15:** VisualizzazioneConfronto
- **UC16:** VisualizzazioneEvoluzione
- **UC17:** MonitoraggioDashboard
- **UC18:** InviaNotifica

Casi d'uso di inclusione / estensione

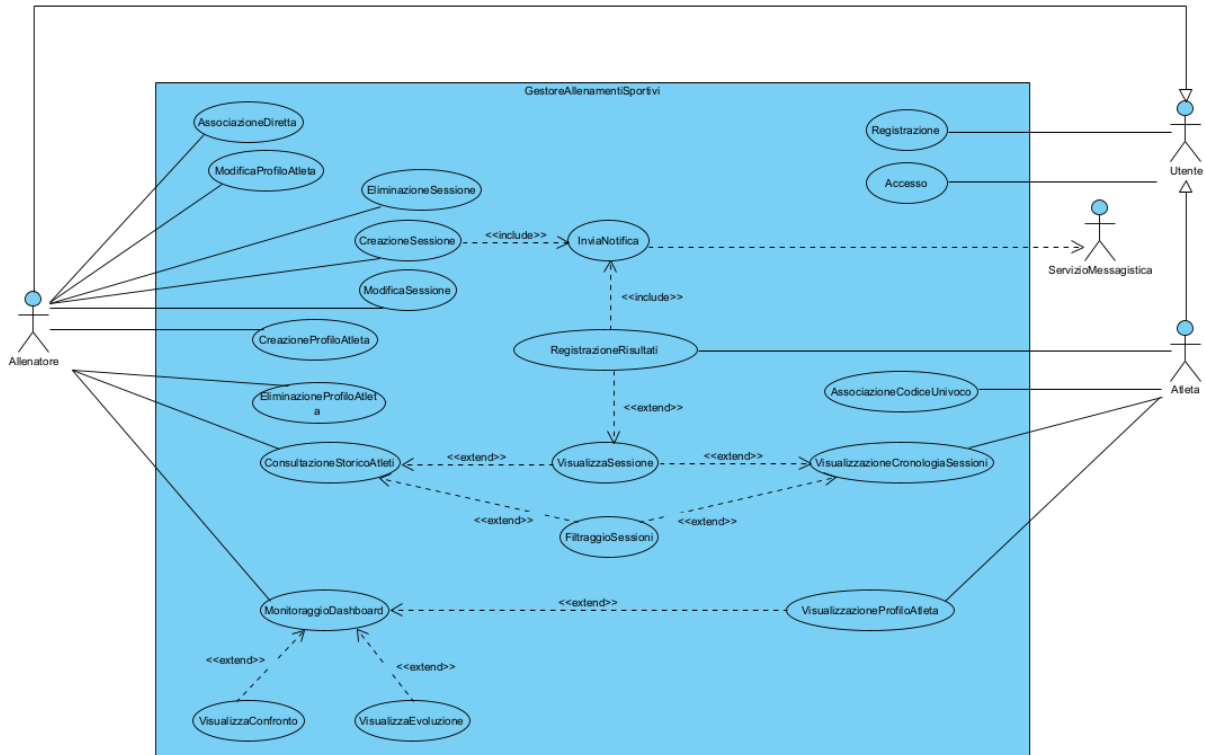
- **UC19:** VisualizzazioneSessione
- **UC20:** FiltraggioSessioni
- **UC21:** VerificaCodiceUnivoco
- **UC22:** VerificaEsistenzaAtleta
- **UC23:** VerificaEsistenzaProfiloAtleta
- **UC24:** VerificaAssociazioneAtleta
- **UC25:** VerificaEsistenzaAllenatore



Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.	Requisiti corrispondenti
UC1: Registrazione	Utente		-	RF01
UC2: Accesso	Utente	-	-	RF02
UC3: Creazione Profilo Atleta	Allenatore	-	Include, Verifica Esistenza Atleta, Verifica Associazione Atleta	RF03
UC4: Eliminazione Profilo Atleta	Allenatore	-	include: Verifica Esistenza Profilo Atleta	RF03
UC5: Modifica Profilo Atleta	Allenatore	-	Include: Verifica Esistenza Profilo Atleta	RF03
UC6: Associazione Diretta	Allenatore	-	Include: <u>Verifica Esistenza Atleta</u> ,	RF04
UC7: Associazione Codice Univoco	Atleta	-	Include: Verifica Codice Univoco, Verifica Esistenza Allenatore	RF05
UC8: Visualizza Profilo Atleta	Atleta			RF13
UC9: Creazione Sessione	Allenatore	-		RF06
UC10: Modifica Sessione	Allenatore	-	-	RF06
UC11: Eliminazione Sessione	Allenatore	-	-	RF06
UC12: Visualizzazione Cronologia Sessioni	Atleta	-		RF07
UC13: Registrazione Risultati	Atleta			RF08
UC14: Consultazione Storico Atleti	Allenatore	-		RF09
UC15: Visualizzazione Confronto	Allenatore			RF10
UC16: Visualizzazione Evoluzione	Allenatore			RF12
UC17: Monitoraggio Dashboard	Allenatore			RF14, RF15
UC18: Invia Notifica	Atleta, Allenatore	Servizio di messaggistica		RF17
UC19: Visualizza Sessione	Atleta, Allenatore		Extend: Visualizzazione Cronologia Sessioni, Consultazione Storico Atleti, Monitoraggio Dashboard	RF11
UC20: Filtraggio Sessioni	Atleta, Allenatore		Extend: Visualizzazione Cronologia Sessioni, Consultazione Storico Atleti, Monitoraggio Dashboard	RF16

2.5.2 Diagramma dei casi d'uso





2.5.3 Scenari

Caso d'uso:	Registrazione
Attore primario	Utente
Attore secondario	-
Descrizione	Un utente si registra al sistema, inserendo le sue informazioni personali e specificando un ruolo tra allenatore e atleta
Pre-Condizioni	-
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Utente accede alla sezione dedicata alla registrazione 2. L'Utente inserisce le sue informazioni personali, quali nome, cognome, data di nascita, e-mail, password e ruolo che avrà nel sistema 3. If le verifiche sulle informazioni sono andate a buon fine 3.1. If l'Utente non si è registrato in precedenza 3.1.1. Il sistema salva le informazioni personali dell'Utente 3.1.2. Il sistema crea l'account associato all'Utente
Post-Condizioni	L'Utente è ora registrato e salvato nel sistema, futuri accessi saranno consentiti e registrazioni con credenziali uguali negate
Casi d'uso correlati	-
Sequenza di eventi alternativi	<u>Al punto 3, se le verifiche non vanno a buon fine, verrà chiesto all'Utente di controllare che le informazioni inserite siano valide.</u> <u>Al punto 3.1, se l'Utente si è già registrato, gli sarà impedito di creare un nuovo account con le stesse credenziali.</u>

Caso d'uso:	Accesso
Attore primario	Utente
Attore secondario	-



Descrizione	Un Utente accede al suo account, usando le sue credenziali che ha usato nella registrazione
Pre-Condizioni	L'Utente si è precedentemente registrato
Sequenza di eventi Principale	1. Il caso d'uso inizia quando l'Utente accede alla sezione dedicata all'accesso 2. L'Utente inserisce le sue credenziali 3. If se le verifiche vanno a buon fine 3.1. If se l'Utente si è registrato in precedenza 3.1.1. Il sistema permette l'accesso alle varie sezioni disponibili, in base al ruolo scelto durante la registrazione
Post-Condizioni	-
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 3, se le verifiche non vanno a buon fine, verrà chiesto all'Utente di controllare che le informazioni inserite siano valide. Al punto 3.1, se l'Utente non si è già registrato, gli sarà impedito di accedere, e richiesto di creare prima un account.

Caso d'uso:	AssociazioneDiretta
Attore primario	Allenatore
Attore secondario	Atleta
Descrizione	L'Allenatore si associa un Atleta, in modo da potere creare un profilo Atleta personalizzato
Pre-Condizioni	L'Allenatore ha effettuato l'accesso L'Atleta ha effettuato la registrazione
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Allenatore accede alla sezione dedicata all'associazione diretta 2. L'Allenatore ricerca un Atleta da associare al suo profilo 3. If se le verifiche vanno a buon fine 3.1. If se l'Atleta ricercato viene trovato 3.1.1. Il sistema permette l'associazione tra Allenatore ed Atleta
Post-Condizioni	L'Allenatore potrà ora gestire le informazioni e le sessioni riguardanti <u>l'Atleta</u> appena associato
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 3, se le verifiche falliscono, verrà chiesto all'Allenatore di verificare la correttezza delle informazioni inserite. Al punto 3.1, se l'Atleta cercato non viene trovato, verrà notificato che non esiste nessun Atleta registrato con quelle credenziali.

Caso d'uso:	AssociazioneCodiceUnivoco
Attore primario	Atleta
Attore secondario	Allenatore
Descrizione	L'Atleta, tramite un codice fornitogli dall'Allenatore, si associa all'Allenatore desiderato.
Pre-Condizioni	L'Atleta ha effettuato l'accesso L'Allenatore ha effettuato la registrazione
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Atleta accede alla sezione dedicata all'associazione tramite codice 2. L'Atleta inserisce il codice univoco 3. If se le verifiche vanno a buon fine 3.1. If se il codice inserito esiste 3.1.1. Il sistema associa l'Atleta all'Allenatore desiderato
Post-Condizioni	L'Allenatore potrà ora gestire le informazioni e le sessioni riguardanti <u>l'Atleta</u> appena associato
Casi d'uso correlati	-



Sequenza di eventi alternativi	Al punto 3, se le verifiche falliscono, verrà chiesto all'Atleta di verificare la correttezza delle informazioni inserite. Al punto 3.1, se il codice univoco non esiste, verrà notificato che non esiste un Allenatore con questo codice univoco.
---------------------------------------	---

Caso d'uso:	RegistrazioneRisultati
Attore primario	Atleta
Attore secondario	ServizioMessagistica
Descrizione	L'Atleta registra nel sistema i risultati ottenuti durante una sessione di allenamento
Pre-Condizioni	L'Atleta ha effettuato l'accesso Esiste almeno una sessione di allenamento attiva per l'Atleta
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Atleta accede alla sezione dedicata alla registrazione dei risultati 2. L'Atleta seleziona una sessione di allenamento attiva 3. <<include>> <i>VisualizzaSessione</i> 4. If la verifica va a buon fine 4.1. L'Atleta inserisce i risultati registrati durante l'allenamento 4.2. Viene mandata una notifica all'Allenatore della registrazione dei risultati 4.3. <<include>> <i>InviaNotifica</i>
Post-Condizioni	Vengono aggiornate le informazioni dei risultati riguardanti la sessione di allenamento dell'Atleta
Casi d'uso correlati	<i>VisualizzaSessione, InviaNotifica</i>
Sequenza di eventi alternativi	Al punto 4, se non esiste nessuna sessione attiva per l'Atleta, viene riportato un messaggio di errore.

Caso d'uso:	CreazioneSessione
Attore primario	Allenatore
Attore secondario	ServizioMessagistica
Descrizione	Un Allenatore crea una nuova sessione di allenamento nel sistema, definita da titolo, descrizione, la data, durata prevista(opzionale), e l'atleta convocato. Al termine della procedura, il sistema genera e invia in automatico una notifica di convocazione all'atleta interessato.
Pre-Condizioni	L'Allenatore ha effettuato correttamente l'accesso al sistema. Almeno un atleta deve essere già registrato ed associato all'allenatore.
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Allenatore accede alla sezione dedicata alla gestione del calendario o alla pianificazione degli allenamenti 2. L'Allenatore richiede la creazione di una nuova sessione. 3. Il sistema mostra un modulo d'inserimento in cui l'Allenatore deve specificare i dettagli dell'allenamento 4. L'Allenatore seleziona un atleta da associare alla sessione attingendo dalla lista dei propri atleti e inserisce i dati relativi alla sessione. 5. L'Allenatore conferma il salvataggio della sessione. 6. if i dati inseriti sono validi 6.1. Il sistema registra permanentemente la nuova sessione nel database, associandovi gli atleti scelti. 6.2. <<include>> <i>InviaNotifica</i> 6.3. Il sistema inoltra la richiesta di invio notifica all'attore esterno ServizioMessagistica per raggiungere i dispositivi dell'atleta convocato



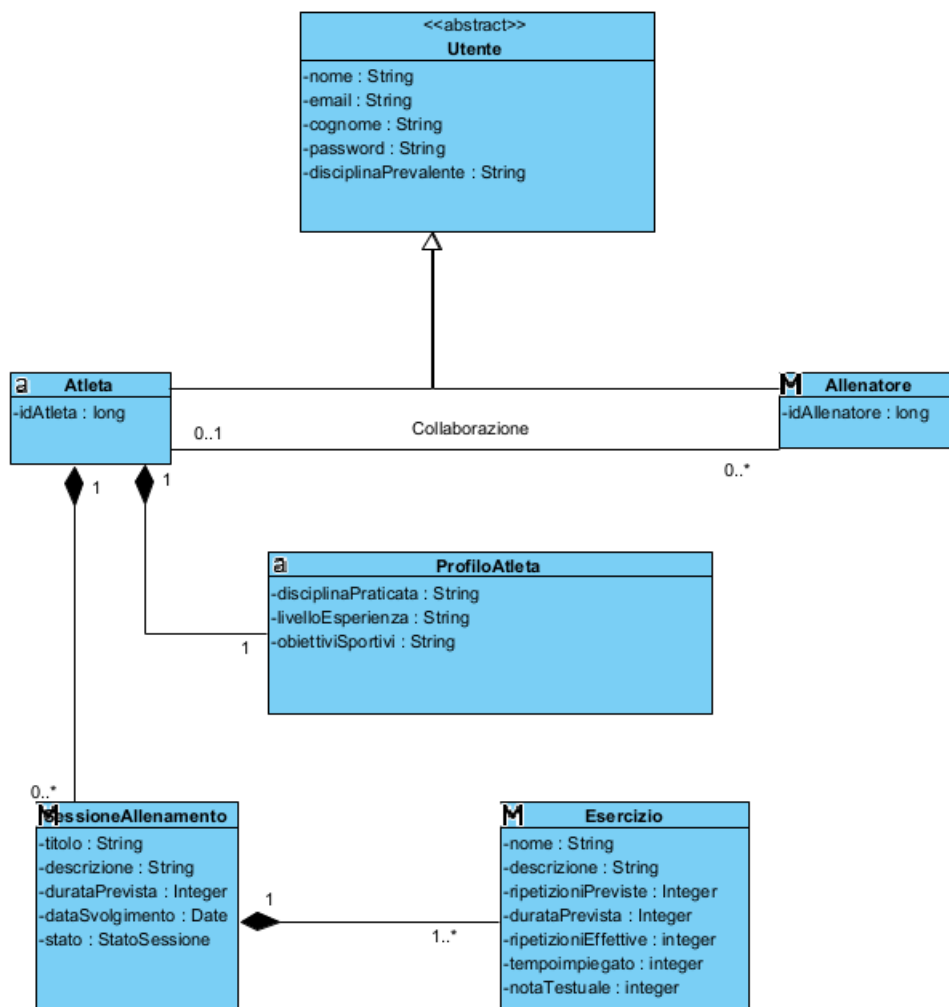
	6.4. Il sistema mostra all'Allenatore un messaggio di conferma dell'avvenuta creazione e pianificazione della sessione.
Post-Condizioni	La sessione di allenamento è memorizzata nello storico del sistema e compare nel calendario degli atleti convocati. Gli atleti hanno ricevuto la notifica della nuova sessione sul proprio account/dispositivo.
Casi d'uso correlati	<i>InviaNotifica, ModificaSessione, EliminazioneSessione</i>
Sequenza di eventi alternativi	Al punto 6, se i dati obbligatori inseriti dall'Allenatore non sono validi (es. data antecedente a quella odierna o obbligatori vuoti), il sistema restituisce un messaggio di errore evidenziando i campi errati e non salva la sessione, rimanendo in attesa delle correzioni. Al punto 6.4, se il ServizioMessaggistica esterno non è momentaneamente raggiungibile o restituisce un errore di rete, il sistema memorizza comunque correttamente la sessione nel database, mostra un messaggio del tipo "Sessione creata, ma l'invio delle notifiche è fallito e verrà riprovato in seguito", e mette in coda la notifica per un invio successivo.

Caso d'uso:	MonitoraggioDashboard
Attore primario	Allenatore
Attore secondario	-
Descrizione	L'allenatore accede alla dashboard per monitorare gli indicatori di continuità e i dati aggregati delle prestazioni dei propri atleti associati. Dalla dashboard ha la possibilità di confrontare i parametri assegnati/effettivi, analizzare l'evoluzione temporale o visualizzare il profilo sportivo completo, di un atleta specifico.
Pre-Condizioni	L'Allenatore ha effettuato l'accesso al sistema.
Sequenza di eventi principale	1. L'Allenatore accede alla sezione dedicata al monitoraggio 2. Il sistema verifica se l'Allenatore ha almeno un atleta associato nel proprio elenco. 3. if l'Allenatore ha atleti associati 3.1. Il sistema calcola e mostra gli indicatori di continuità e i dati aggregati relativi agli atleti (RF14). 3.2. L'Allenatore seleziona un atleta specifico dall'elenco per analizzarne i dettagli. 3.3. Il sistema mostra le opzioni di monitoraggio avanzato per l'atleta selezionato. 3.4. if l'Allenatore decide di verificare lo scostamento tra pianificazione ed esecuzione 3.4.1. L'Allenatore richiede il confronto dei parametri. 3.4.2. if l'atleta ha registrato almeno una prestazione 3.4.2.1. Il sistema mostra il confronto tra i parametri pianificati e i risultati reali (RF10). 3.4.2.2. <<extend>> VisualizzaConfronto 3.5. if l'Allenatore decide di analizzare l'andamento nel tempo 3.5.1. L'Allenatore richiede lo storico evolutivo. 3.5.2. if l'atleta ha uno storico di prestazioni registrate 3.5.2.1. Il sistema elabora e mostra i grafici relativi all'evoluzione delle prestazioni (RF12). 3.5.2.2. <<extend>> VisualizzaEvoluzione 3.6. if l'Allenatore decide di esaminare i dati personali e gli obiettivi dell'atleta 3.6.1. L'Allenatore richiede i dettagli del profilo dell'atleta. 3.6.2. Il sistema mostra il profilo sportivo completo dell'atleta selezionato (RF15). 4.4. 3.6.3. <<extend>> VisualizzaProfiloAtleta



Post-Condizioni	L'Allenatore ha consultato i dati aggregati e specifici sulle performance e sullo stato degli atleti senza alterarne i dati memorizzati.
Casi d'uso correlati	<i>VisualizzaConfronto, VisualizzaEvoluzione, VisualizzaProfiloAtleta</i>
Sequenza di eventi alternativi	Al punto 3: Se l'Allenatore non ha alcun atleta associato, il sistema restituisce un messaggio di avviso Al punto 3.4.2: Se l'atleta selezionato non ha ancora dati di allenamento registrati, il sistema mostra un messaggio di errore ("Nessun dato registrato per questo atleta. Impossibile effettuare il confronto"). Al punto 3.5.2: Se l'atleta non ha uno storico di prestazioni, il sistema mostra un messaggio di errore ("Dati insufficienti per generare il grafico evolutivo").

2.6 Diagramma delle classi



RESPONSABILITÀ	CLASSE
<i>Registrazione</i>	GestoreUtenti
<i>Accesso</i>	GestoreUtenti



<i>InizializzazioneProfiloAtleta</i>	Atleta
<i>EliminazioneProfiloAtleta</i>	Atleta
<i>ModificaProfiloAtleta</i>	Atleta
<i>AssociazioneCodiceUnivoco</i>	GestoreUtenti
<i>AssociazioneDiretta</i>	GestoreUtenti
<i>VisualizzaProfiloAtleta</i>	Atleta
<i>CreazioneSessione</i>	Atleta
<i>ModificaSessione</i>	Atleta
<i>EliminazioneSessione</i>	Atleta
<i>RegistrazioneRisultati</i>	Esercizio
<i>VisualizzazioneCronologiaSessioni</i>	GestoreSessioni
<i>FiltraggioSessioni</i>	GestoreSessioni
<i>VisualizzaSessione</i>	SessioneAllenamento
<i>MonitoraggioDashboard</i>	GestoreDashboard
<i>VisualizzaConfronto</i>	GestoreDashboard
<i>VisualizzaEvoluzione</i>	GestoreDashboard

Registrazione, Login, AssociazioneAllenatoreAtleta sono responsabilità di GestoreUtenti. Invece di assegnare la complessa logica direttamente alle entità di dominio come Atleta o Allenatore abbiamo scelto di astrarre queste operazioni in una classe dedicata. GestoreUtenti riceve e gestisce le richieste di accesso dall'interfaccia come un <use case controller>. Inoltre, GestoreUtenti agisce come <creator> per le classi Atleta e Allenatore (sottoclassi di Utente). Secondo le linee guida GRASP

*InizializzazioneProfiloAtleta, EliminazioneProfilo, ModificaProfiloAtleta, VisualizzaProfiloAtleta sono responsabilità di **Atleta**, perché **Atleta** è composta dalla classe ProfiloAtleta. Essendo il "contenitore" e gestore del proprio profilo, l'Atleta possiede tutte le informazioni necessarie<<information expert>>e <<creator>>.*

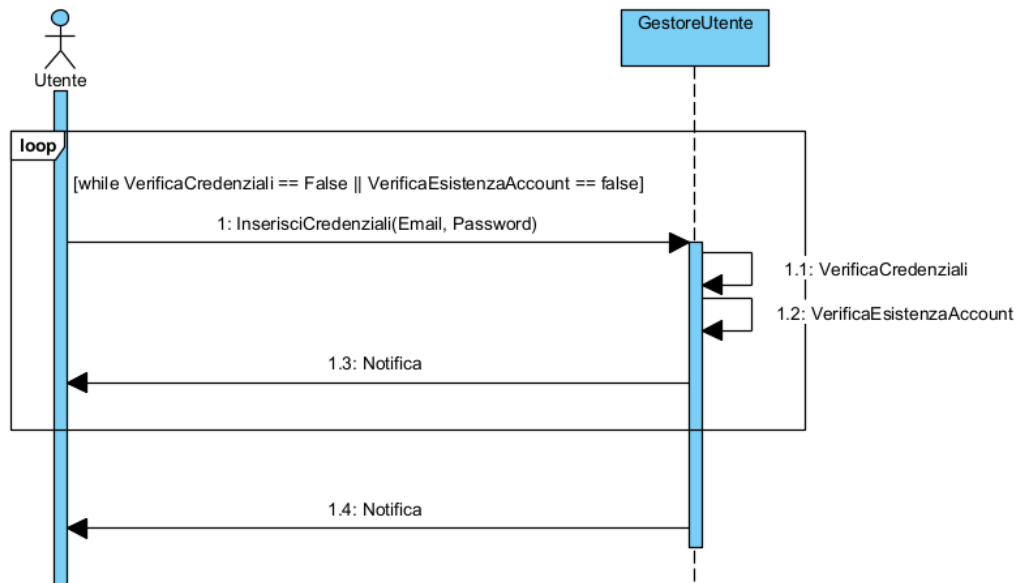
*CreazioneSessione, ModificaSessione, EliminazioneSessione sono sempre responsabilità dell'**Atleta** per gli stessi motivi citati per il profilo, <<creator>> e information expert>>*

*VisualizzazioneCronologiaSessioni, FiltraggioSessioni sono responsabilità di **GestoreSessioni**. Raccogliere, filtrare e mostrare la cronologia richiede di operare su un insieme di sessioni. Invece di delegare questa logica complessa all'Atleta (che diminuirebbe la sua coesione facendogli fare troppe cose), abbiamo scelto di creare un GestoreSessioni ad hoc<<use case controller>>. Questo mantiene il sistema disaccoppiato e coeso.*

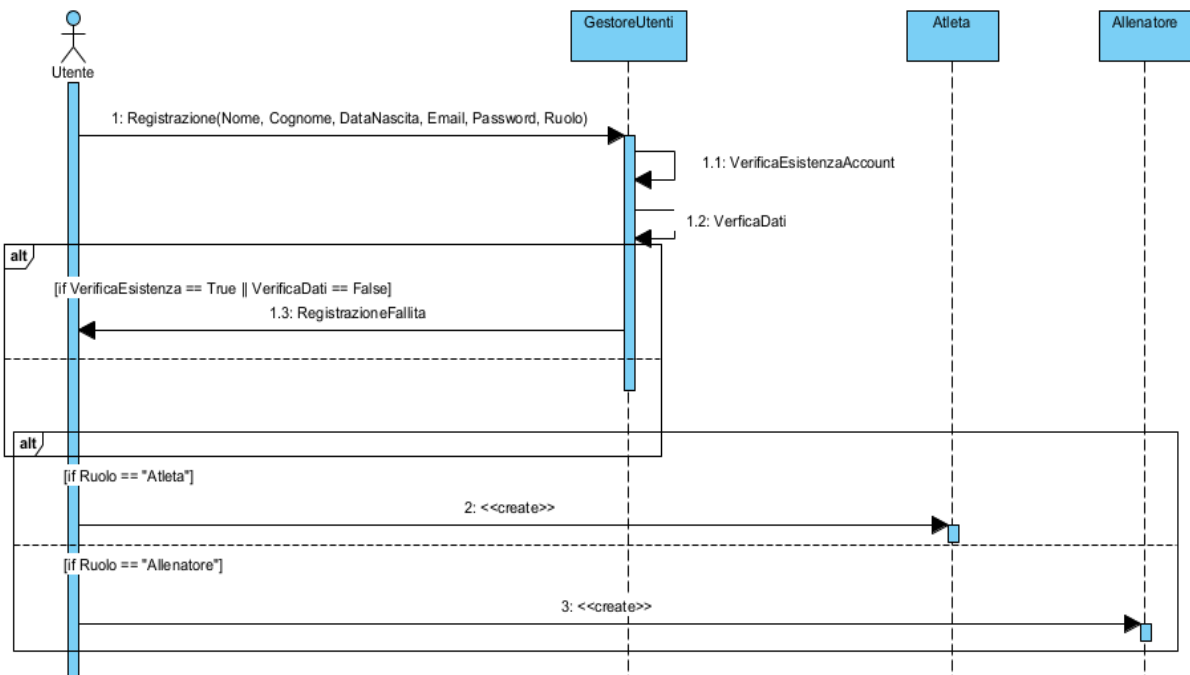
*MonitoraggioDashboard, VisualizzaConfronto, VisualizzaEvoluzione sono responsabilità di **GestoreDashboard**. Funge da punto di raccolta per il calcolo degli indicatori aggregati << use case controller>> separare le logiche per la creazione delle specifiche statistiche e dati per l'allenatore in una classe dedicata garantisce un **Basso Accoppiamento** e un'**Alta Coesione***

2.7 Diagrammi di sequenza

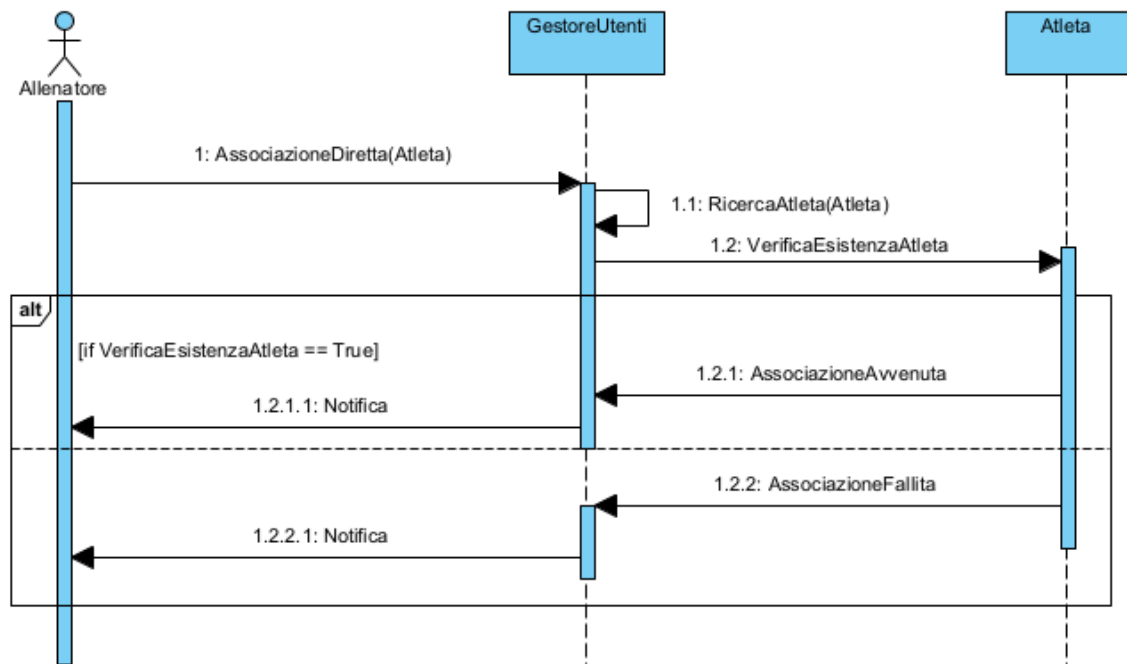
2.7.1 Accesso



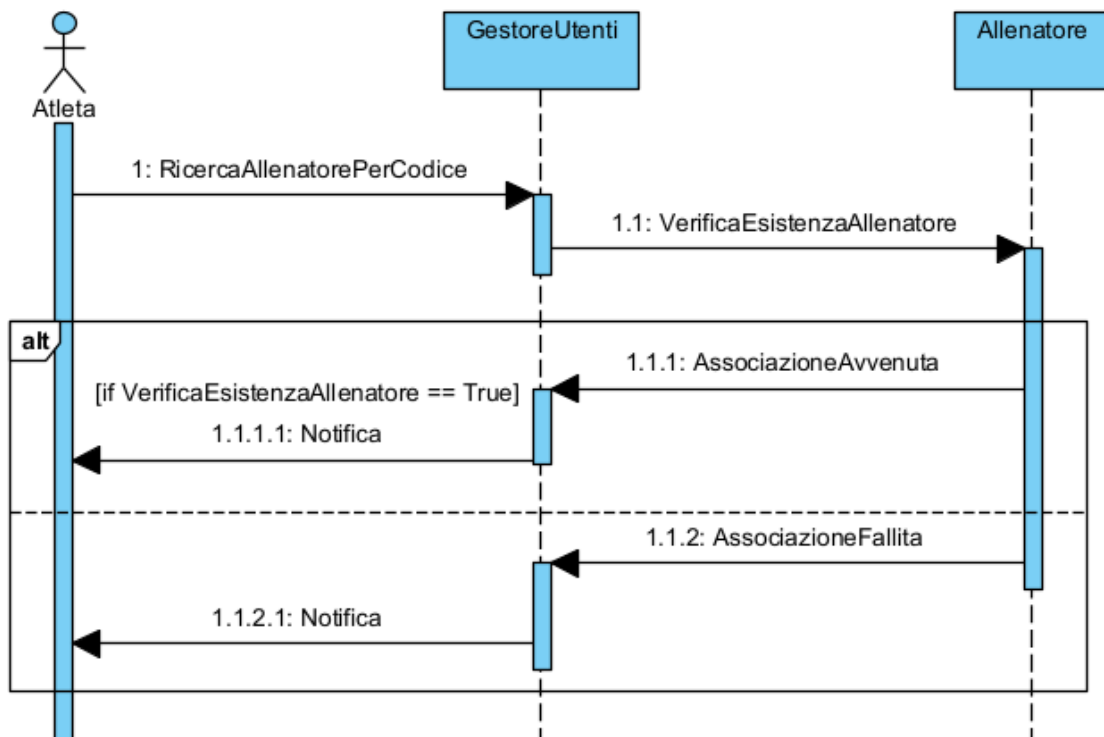
2.7.2 Registrazione



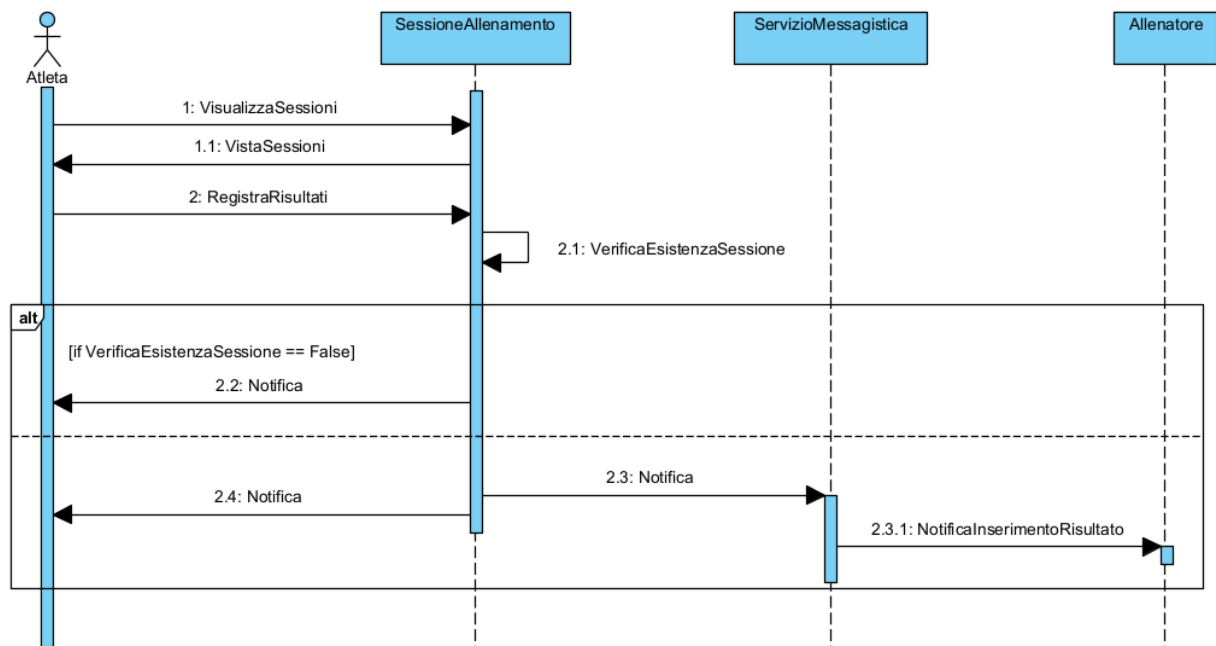
2.7.3 AssociazioneDiretta



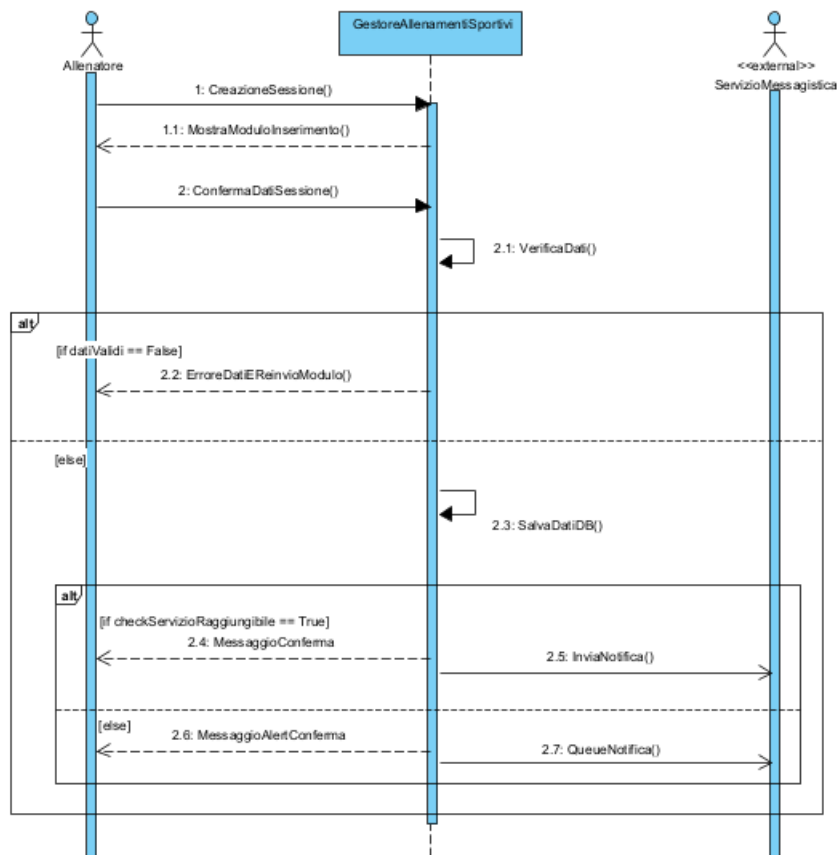
2.7.4 AssociazioneCodiceUnivoco



2.7.5 RegistrazioneRisultati

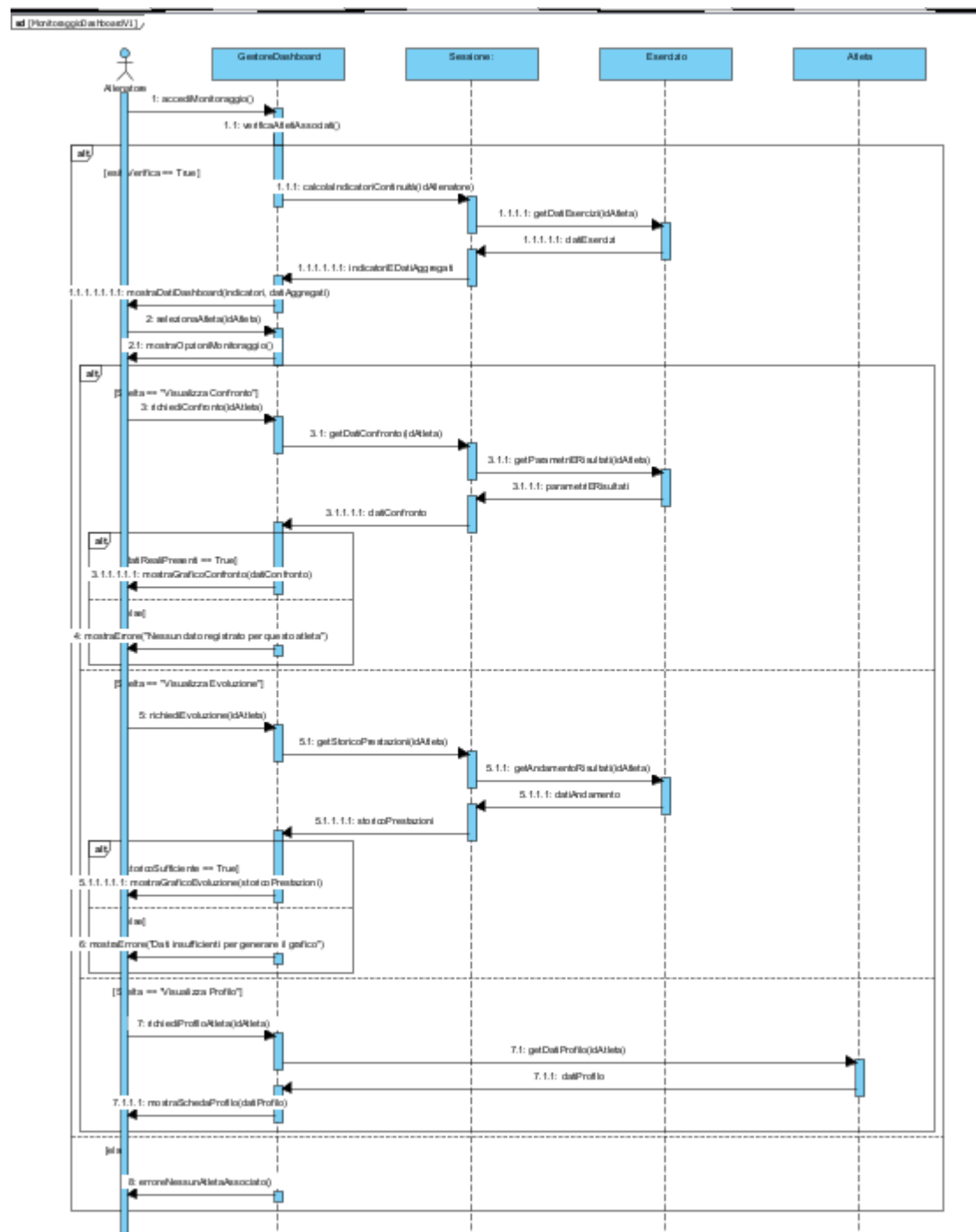


2.7.6 CreazioneSessione



2.7.7 MonitoraggioDashboard





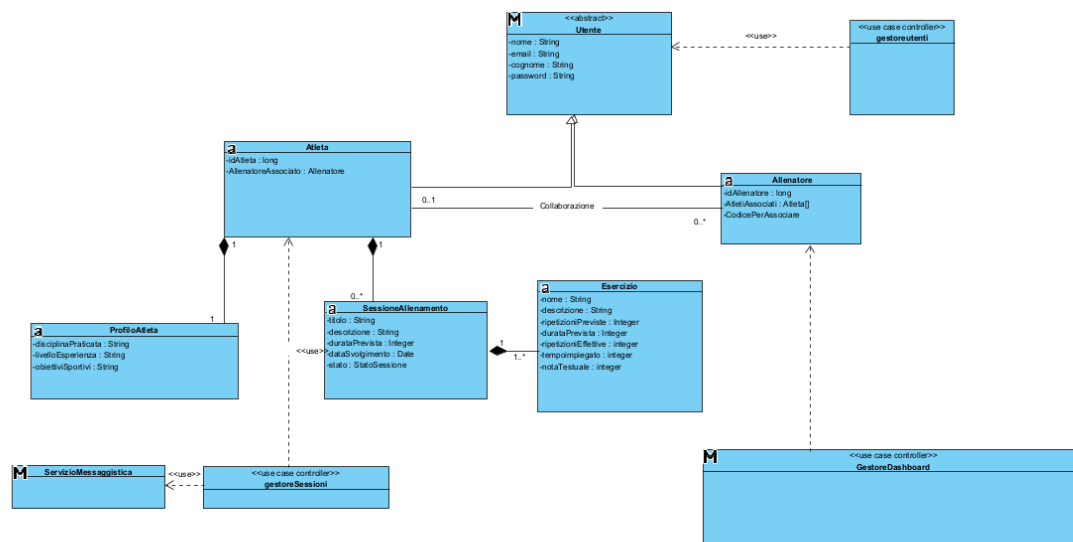
2.8 Diagramma delle classi raffinato

Le aggiunte e le modifiche fatte nel corso della costruzione dei Sequence Diagrams hanno determinato lo sviluppo di un **Diagramma delle Classi** raffinato che riporta maggiori dettagli sugli attributi e le principali operazioni delle classi: Durante la fase di modellazione dinamica, passando da una visione puramente concettuale a un design più vicino all'implementazione (in ottica di architettura BCED), è emersa chiaramente la necessità di migliorare la separazione dei compiti. Ci siamo accorti, infatti, che delegare le logiche di coordinamento e l'elaborazione degli eventi di sistema direttamente alle classi di dominio (come *Atleta*, *Allenatore* o



SessioneAllenamento) avrebbe compromesso la qualità del design, rendendo queste classi troppo complesse e poco focalizzate sul loro scopo principale.

Per questo motivo, si è reso necessario introdurre delle apposite classi di controllo, ovvero i "gestori" (come *GestoreUtenti*, *GestoreSessioni* e *GestoreDashboard*). Questa scelta è stata guidata dalla volontà di aumentare la coesione del sistema e di rispettare rigorosamente i pattern GRASP per l'assegnamento delle responsabilità. Applicando in particolare il pattern *Controller*, abbiamo affidato a queste nuove classi il compito di orchestrare le operazioni, mantenendo le classi Entity leggere, altamente coese e focalizzate esclusivamente sulla gestione dei dati legati all'allenamento, garantendo così un'architettura complessivamente più robusta e manutenibile.



3. Piano di test funzionale

Si intende progettare i casi di test funzionale con la tecnica del *Category Partition Testing*.

3.1 Registrazione

REGISTRAZIONE						
NOME	COGNOME	PASSWORD	EMAIL	DISCIPLINA	RUOLO	CODICE ASSOCIAZIONE
- Stringa non vuota (Classe Valida) - Stringa vuota, con soli spazi o nulla [ERROR]	- Stringa non vuota (Classe Valida) - Stringa vuota o nulla [ERROR]	- Stringa non vuota di lunghezza ≥ 6 (Classe Valida per tutti) - Stringa di lunghezza < 6 (Valida per Atleta / [ERROR])	- Stringa non vuota contenente il carattere @ (Classe Valida) - Stringa non contenente il carattere @ [ERROR]	- Stringa non vuota (Classe Valida) - Stringa vuota o nulla [ERROR]	- Valore selezionato pari a "Atleta" (Valido) - Valore selezionato pari a "Allenatore" (Valido) - Nessun valore selezionato /	- Stringa vuota (Valido solo per Atleta) - Codice esistente nel sistema (Valido per Atleta / [ERROR] per Allenatore perché già in uso)

		per Allenatore) • Stringa vuota o nulla [ERROR]	• Stringa vuota o nulla [ERROR]		"Seleziona... " o nullo [ERROR] •	• Codice non esistente nel sistema (Valido per Allenatore / [ERROR] per Atleta perché non trovato) • Stringa vuota [ERROR] (per Allenatore - codice obbligatorio)
--	--	---	---------------------------------	--	--------------------------------------	---

Il numero di test da effettuarsi senza particolari vincoli è: $3 \times 3 \times 4 \times 3 \times 2 \times 3 \times 3 = 1944$. Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è 14 (2 per Nome, 2 per Cognome, 3 per Password, 2 per E-mail, 1 per Disciplina, 1 per Ruolo, 3 per Codice Associazione).

Il numero di test risultante è: $3 + 14 = 17$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi (Atleta, senza codice)	Nome valido, Cognome valido, Password valida, E-mail valida, Disciplina valida, Ruolo "Atleta" valido, Codice vuoto (Valido per Atleta)		"{Nome: ""Mario"", Cognome: ""Rossi"", Password: ""Pass123"", E-mail: ""mario.rossi@email.it"", Disciplina: ""Nuoto"", Ruolo: ""Atleta"", CodiceAssociazione: """"}"	"Registrazione Atleta completata!"	L'Atleta viene registrato con successo nel sistema, tutti i campi della GUI vengono puliti.
2	Atleta registrato con Codice Allenatore esistente	Nome valido, Cognome valido, Password valida, E-mail valida, Disciplina valida, Ruolo "Atleta" valido, Codice esistente nel sistema (Valido per Atleta)	Il codice "COACH100" è già presente e registrato nel sistema.	"{Nome: ""Mario"", Cognome: ""Rossi"", Password: ""Pass123"", E-mail: ""mario.rossi@email.it"", Disciplina: ""Nuoto"", Ruolo: ""Atleta"", CodiceAssociazione: ""COACH100""}"	"Registrazione Atleta completata!"	L'Atleta viene registrato nel sistema e associato formalmente all'allenatore avente codice "COACH100". Campi puliti.
3	Nome vuoto	Nome vuoto [ERROR], tutti gli altri validi		"{Nome: """, Cognome: ""Rossi"", Password: ""Pass123"", E-mail: ""mario.rossi@email.it"", Disciplina: ""Nuoto"", Ruolo: ""Atleta"", CodiceAssociazione: """"}"	"Errore: compila tutti i campi base."	Nessun inserimento nel sistema. La GUI mostra il messaggio di errore in rosso.



4	Cognome vuoto	Cognome vuoto [ERROR], tutti gli altri validi		"{Nome: ""Mario"", Cognome: """, Password: ""Pass123"", E-mail: ""mario.rossi@email.it"", Disciplina: ""Nuoto"", Ruolo: ""Atleta"", CodiceAssociazione: """}"	"Errore: compila tutti i campi base."	Nessun inserimento nel sistema. La GUI mostra il messaggio di errore in rosso.
5	Password vuota	Password vuota [ERROR], tutti gli altri validi		"{Nome: ""Mario"", Cognome: ""Rossi"", Password: """, E-mail: ""mario.rossi@email.it"", Disciplina: ""Nuoto"", Ruolo: ""Atleta"", CodiceAssociazione: """}"	"Errore: compila tutti i campi base."	Nessun inserimento nel sistema. La GUI mostra il messaggio di errore in rosso.
6	Disciplina vuota	Disciplina vuota [ERROR], tutti gli altri validi		"{Nome: ""Mario"", Cognome: ""Rossi"", Password: ""Pass123"", E-mail: ""mario.rossi@email.it"", Disciplina: """, Ruolo: ""Atleta"", CodiceAssociazione: """}"	"Errore: compila tutti i campi base."	Nessun inserimento nel sistema. La GUI mostra il messaggio di errore in rosso.
7	E-mail vuota	E-mail vuota [ERROR], tutti gli altri validi		"{Nome: ""Mario"", Cognome: ""Rossi"", Password: ""Pass123"", E-mail: """, Disciplina: ""Nuoto"", Ruolo: ""Atleta"", CodiceAssociazione: """}"	"Errore: compila tutti i campi base."	Nessun inserimento nel sistema. La GUI mostra il messaggio di errore in rosso.
8	E-mail senza carattere '@' (Atleta)	E-mail senza '@' [ERROR], tutti gli altri validi		"{Nome: ""Mario"", Cognome: ""Rossi"", Password: ""Pass123"", E-mail: ""mario.rossiemail.it"", Disciplina: ""Nuoto"", Ruolo: ""Atleta"", CodiceAssociazione: """}"	"Errore: Dati obbligatori mancanti o errati."	Eccezione sollevata dal backend (GestoreUtenti). Nessuna registrazione.
9	E-mail senza carattere '@' (Allenatore)	E-mail senza '@' [ERROR], tutti gli altri validi		"{Nome: ""Claudio"", Cognome: ""Verdi"", Password: ""PassSecure123"", E-mail: ""claudio.verdiemail.it"", Disciplina: ""Tennis"", Ruolo: ""Allenatore"", CodiceAssociazione: ""COACH200""}"	"Errore: Dati anagrafici mancanti o formato email errato."	Eccezione sollevata dal backend (GestoreUtenti). Nessuna registrazione.
10	Password troppo corta per Allenatore (<6 caratteri)	Password < 6 per Allenatore [ERROR], tutti gli altri validi		"{Nome: ""Claudio"", Cognome: ""Verdi"", Password: ""12345"", E-mail: ""claudio@email.it"", Disciplina: ""Tennis"", Ruolo: ""Allenatore"", CodiceAssociazione: ""COACH200""}"	"Errore: La password deve contenere almeno 6 caratteri."	Eccezione sollevata dal backend. Nessun utente salvato.
11	Nome con soli spazi bianchi (Allenatore)	Nome vuoto/con spazi [ERROR], tutti gli altri validi		"{Nome: """, Cognome: ""Verdi"", Password: ""PassSecure123"", E-mail: ""claudio@email.it"",	"Errore: Dati anagrafici mancanti o	Validazione isBlank() fallita a livello di



				Disciplina: ""Tennis"", Ruolo: ""Allenatore"", CodiceAssociazione: ""COACH200""}	formato email errato."	controller per l'Allenatore .
12	Ruolo non selezionato / nullo	Ruolo non selezionato [ERROR]		"{Nome: ""Mario"", Cognome: ""Rossi"", Password: ""Pass123"", E-mail: ""mario@email.it"", Disciplina: ""Nuoto"", Ruolo: ""Seleziona..."", CodiceAssociazione: """"}"	"Errore: Seleziona Atleta o Allenatore."	Controllo GUI fallito, interruzione e dell'esecuzione.
13	Atleta con Codice Allenatore non esistente	Codice non esistente (Atleta) [ERROR], tutti gli altri validi	Il codice "COACH_INV" non è associato a nessun allenatore nel sistema.	"{Nome: ""Mario"", Cognome: ""Rossi"", Password: ""Pass123"", E-mail: ""mario@email.it"", Disciplina: ""Nuoto"", Ruolo: ""Atleta"", CodiceAssociazione: ""COACH_INV""}"	"Errore: Il codice allenatore inserito non esiste."	Controllo d'esistenza fallito. Nessun atleta inserito nel sistema.
14	Allenatore con Codice Associazione vuoto	Codice vuoto (Allenatore) [ERROR], tutti gli altri validi		"{Nome: ""Claudio"", Cognome: ""Verdi"", Password: ""PassSecure123"", E-mail: ""claudio@email.it"", Disciplina: ""Tennis"", Ruolo: ""Allenatore"", CodiceAssociazione: """"}"	"Errore: Inserire Codice Associazione."	Il codice per l'allenatore è obbligatorio, la GUI blocca l'operazione.
15	Allenatore con Codice Associazione già esistente	Codice già esistente (Allenatore) [ERROR], tutti gli altri validi	Il codice "COACH100" è già associato a un allenatore registrato.	"{Nome: ""Claudio"", Cognome: ""Verdi"", Password: ""PassSecure123"", E-mail: ""claudio@email.it"", Disciplina: ""Tennis"", Ruolo: ""Allenatore"", CodiceAssociazione: ""COACH100""}"	"Errore: Questo codice è già in uso da un altro allenatore."	Controllo di univocità fallito. Registrazione respinta.
16	Tutti input validi (Allenatore)	Nome valido, Cognome valido, Password ≥ 6 , E-mail valida, Disciplina valida, Ruolo "Allenatore" valido, Codice nuovo (Valido per Allenatore)	Il codice "COACH300" non è in uso da nessun altro allenatore.	"{Nome: ""Claudio"", Cognome: ""Verdi"", Password: ""PassSecure123"", E-mail: ""claudio@email.it"", Disciplina: ""Tennis"", Ruolo: ""Allenatore"", CodiceAssociazione: ""COACH300""}"	"Registrazione Allenatore completata!"	L'Allenatore viene registrato con successo. I campi della GUI vengono puliti.



3.2 Accesso

Accesso	
PASSWORD	EMAIL
- Stringa di lunghezza ≥ 6 (<i>Classe Valid</i>) - Stringa di lunghezza < 6 [ERROR] - Stringa vuota o nulla [ERROR]	- Stringa contenente il carattere "@" (<i>Classe Valid</i>) - Stringa non contenente il carattere "@" [ERROR] - Stringa vuota o nulla [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: $3 \times 3 = 9$. Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è 4 (2 per Password, 2 per E-mail). Il numero di test risultante è: $(1 \times 1) + 4 = 5$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	Password valida (\$\geq 6\$), E-mail valida (contiene @)		{Password: "Password1!", E-mail: "mario.rossi@email.it"}	Tutti input validi	Si ricevono le credenziali per e-mail
2	Password troppo breve (\$< 6\$ caratteri)	Password \$< 6\$ [ERROR], E-mail valida		{Password: "Pass1", E-mail: "mario.rossi@email.it"}	Password troppo breve (\$< 6\$ caratteri)	
3	Password vuota	Password vuota [ERROR], E-mail valida		{Password: "", E-mail: "mario.rossi@email.it"}	Password vuota	
4	E-mail senza carattere speciale @	E-mail senza @ [ERROR], Password valida		{Password: "Password1!", E-mail: "mario.rossiemailit"}	E-mail senza carattere speciale @	
5	E-mail vuota	E-mail vuota [ERROR], Password valida		{Password: "Password1!", E-mail: ""}	E-mail vuota	

3.3 Associazione Diretta

REGISTRAZIONE
IDATLETA



- Numero intero positivo valido (compreso nel range del tipo Long)
- Numero intero < 0 **[ERROR]**
- Stringa contenente simboli non numerici (*NumberFormatException*) **[ERROR]**
- Stringa vuota o composta solo da spazi bianchi **[ERROR]**

Il numero di test da effettuarsi senza particolari vincoli è: 4 . Con i vincoli **[ERROR]**, invece, il numero di test da eseguire per testare singolarmente i vincoli è 3 (3 per IdAllenatore). Il numero di test risultante è: (1) + 3 = 4.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	IdAtleta valido		{IdAtleta: "5467"}	Atleta associato	Si riceve la conferma via e-mail dell'avvenuta associazione
2	IdAtleta < 0	IdAtleta < 0 [ERROR]		{IdAtleta: "-435"}	IdAtleta non valido!	
3	IdAtleta non contiene solo numeri	IdAtleta non contiene solo numeri [ERROR]		{IdAtleta: "46p7"}	Formato IdAtleta non valido	
4	IdAtleta vuoto	IdAtleta vuoto [ERROR]		{IdAtleta: ""}	Campo IdAtleta obbligatorio!	

3.4 AssociazioneCodiceUnivoco

ASSOCIAZIONECODICEUNIVOCO	
CODICEUNIVOCO	
• Stringa non vuota • Stringa vuota [ERROR] • Stringa contenente solo spazi [ERROR] • Valore null [ERROR]	

Il numero di test da effettuarsi senza particolari vincoli è: 4. Con i vincoli **[ERROR]**, invece, il numero di test da eseguire per testare singolarmente i vincoli è 3 (3 per CodiceUnivoco). Il numero di test risultante è: (1) + 3 = 4.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	CodiceUnivoco valido		{CodiceUnivoco: "5467"}	Allenatore associato	Si riceve la conferma via e-mail dell'avvenuta associazione
2	CodiceUnivoco vuoto	CodiceUnivoco vuoto [ERROR]		{CodiceUnivoco: ""}	Codice Univoco obbligatorio!	
3	CodiceUnivoco con solo spazi	CodiceUnivoco con solo spazi [ERROR]		{CodiceUnivoco: " "}	Codice Univoco non valido!	
4	CodiceUnivoco null	CodiceUnivoco null [ERROR]		{CodiceUnivoco: "null"}	Formato Codice Univoco non valido!	

3.5 RegistrazioneRisultati

REGISTRAZIONERISULTATI



RIPETIZIONE EFFETTIVE	TEMPO IMPIEGATO	NOTA
- Numero decimale ≤ 100 - Numero decimale < 0 [ERROR] - Numero che contiene simboli che non sono numeri [ERROR]	- Numero decimale ≤ 120 - Numero decimale < 0 [ERROR] - Numero che contiene simboli che non sono numeri [ERROR]	- Stringa di caratteri alfanumerici e speciali di lunghezza ≤ 1000 - Stringa di caratteri alfanumerici e speciali di lunghezza > 1000 [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: $3 \times 3 \times 2 = 18$. Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è 5 (2 per RipetizioniEffettive, 2 per TempoImpiegato, 1 per Nota). Il numero di test risultante è: $(1 \times 1 \times 1) + 5 = 6$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	Titolo valido, Descrizione valida, TempoPrevisto valido		{Titolo: "Allenamento del 09/06/2026", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.", TempoPrevisto: "90"}	Sessione creata!	L'Allenatore riceve un'e-mail di conferma della creazione della sessione, l'Atleta riceve una notifica della nuova sessione
2	Titolo > 40	Titolo > 40 [ERROR], tutti gli altri validi		{Titolo: "Alleeeneenaaaaaaaaam entoooooooooooo del 09/06/2026", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.", TempoPrevisto: "90"}	Titolo troppo lungo!	
3	Titolo vuoto	Titolo vuoto [ERROR], tutti gli altri validi		{Titolo: "", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.", TempoPrevisto: "90"}	Campo Titolo obbligatorio!	
4	Descrizione > 1000	Descrizione > 1000 [ERROR], tutti gli altri validi		{Titolo: "Allenamento del 09/06/2026", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.", TempoPrevisto: "90"} (TROPPO SPAZIO IMPIEGATO)	Descrizione troppo lunga!	
5	TempoPrevisto < 0	TempoPrevisto < 0 [ERROR], tutti gli altri validi		{Titolo: "Allenamento del 09/06/2026", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.", TempoPrevisto: "-45"}	TempoPrevisto non valido!	
6	TempoPrevisto con simboli/lettere	TempoPrevisto con simboli/lettere [ERROR], tutti gli altri validi		{Titolo: "Allenamento del 09/06/2026", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.", TempoPrevisto: "9ù"}	Formato TempoPrevisto non valido!	

3.6 Monitoraggio Dashboard



MonitoraggioDashboard		
SELEZIONE ATLETA (ComboBox)	STATO DATI EVOLUZIONE	STATO DATI CONFRONTO
Indice > 0 (Selezione di un atleta valido associato) <i>(Classe Valida)</i>	Lo storico delle prestazioni dell'atleta esiste ed è calcolabile <i>(Classe Valida)</i>	I parametri previsti ed effettivi dell'atleta esistono e sono coerenti <i>(Classe Valida)</i>
Indice <= 0 ("Seleziona un atleta...") [ERROR]	Storico prestazioni mancante o insufficiente (IllegalStateException) [ERROR]	Parametri insufficienti per generare lo scostamento (IllegalArgumentException o IllegalStateException) [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: $2 * 2 * 2 = 8$. Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è 3 (1 per Selezione Atleta, 1 per Stato Dati Evoluzione, 1 per Stato Dati Confronto). Il numero di test risultante è: $(1 * 1 * 1) + 3 = 4$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Visualizzazione Profilo valida	Selezione Atleta: Indice > 0	Allenatore loggato con dati caricati. Atleta regolarmente presente nel DB.	Selezione: "1 - Mario Rossi". Click su "Visualizza profilo"	Apertura della schermata GuiProfiloAtleta popolata con i dati dell'atleta.	GuiProfiloAtleta visibile in primo piano.
2	Visualizzazione Evoluzione valida	Selezione Atleta: Indice > 0. Dati Evoluzione: Storico presente	Atleta regolarmente presente nel DB con storico sessioni salvato.	Selezione: "1 - Mario Rossi". Click su "Visualizza evoluzione"	Apertura della schermata GuiEvoluzione con grafico generato.	GuiEvoluzione visibile. Nessuna modifica ai dati.
3	Visualizzazione Confronto valida	Selezione Atleta: Indice > 0. Dati Confronto: Parametri presenti	Atleta regolarmente presente nel DB con dati di scostamento calcolabili.	Selezione: "1 - Mario Rossi". Click su "Visualizza confronto"	Apertura della schermata GuiConfronto con i dati elaborati.	GuiConfronto visibile. Nessuna modifica ai dati.
4	Errore Selezione Mancante	Selezione Atleta: Indice <= 0 [ERROR]	Dashboard aperta, tendina sul valore di default.	Selezione: "Seleziona un atleta...". Click su uno qualsiasi dei tre bottoni.	Viene mostrato un JOptionPane con il messaggio: "Seleziona un atleta dalla lista."	La Dashboard rimane attiva in attesa di selezione. Le altre schermate non si aprono.
5	Errore Dati Evoluzione Insufficienti	Selezione Atleta: Indice > 0. Dati Evoluzione: Mancanti [ERROR]	Atleta selezionato dalla tendina non ha completato sessioni. Il gestore lancia IllegalStateException.	Selezione: "2 - Luigi Verdi". Click su "Visualizza evoluzione"	Viene mostrato un JOptionPane di avviso (WARNING_MESSAGE) con il messaggio dell'eccezione (es. "Dati insufficienti").	La finestra GuiEvoluzione non viene istanziata. Dashboard attiva.
6	Errore Dati Confronto Insufficienti	Selezione Atleta: Indice > 0. Dati Confronto:	L'atleta selezionato non ha i requisiti per il calcolo. Il gestore lancia eccezione.	Selezione: "2 - Luigi Verdi". Click su "Visualizza confronto"	Viene mostrato un JOptionPane di avviso (WARNING_MESSAGE) con il	La finestra GuiConfronto non viene istanziata. Dashboard attiva.



		Mancanti [ERROR]			messaggio dell'eccezione.	
--	--	---------------------	--	--	------------------------------	--

3.7 CreazioneSessione

CREAZIONESESSIONE		
TITOLO	DESCRIZIONE	TEMPOPREVISTO
- Stringa di caratteri alfanumerici e speciali di lunghezza ≤ 40 - Stringa di caratteri alfanumerici e speciali di lunghezza > 40 [ERROR] - Stringa vuota [ERROR]	- Stringa di caratteri alfanumerici e speciali di lunghezza ≤ 1000 - Stringa di caratteri alfanumerici e speciali di lunghezza > 1000 [ERROR]	- Numero decimale ≤ 120 - Numero decimale < 0 [ERROR] - Numero che contiene simboli che non sono numeri [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: $3*2*3 = 18$. Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è 5 (2 per Titolo, 1 per Descrizione, 2 per TempoPrevisto). Il numero di test risultante è: $(1*1*1) + 5 = 6$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	Titolo valido, Descrizione valida, TempoPrevisto valido		{Titolo: "Allenamento del 09/06/2026", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.", TempoPrevisto: "90"}	Sessione creata!	L'Allenatore riceve un'email di conferma della creazione della sessione, l'Atleta riceve una notifica della nuova sessione
2	Titolo > 40	Titolo > 40 [ERROR], tutti gli altri validi		{Titolo: "Alleeeeeeeeeeenaaaaaaam entoooooooooooo del 09/06/2026", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.", TempoPrevisto: "90"}	Titolo troppo lungo!	
3	Titolo vuoto	Titolo vuoto [ERROR], tutti gli altri validi		{Titolo: "", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.", TempoPrevisto: "90"}	Campo Titolo obbligatorio!	
4	Descrizione > 1000	Descrizione > 1000 [ERROR], tutti gli altri validi		{Titolo: "Allenamento del 09/06/2026", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi " , TempoPrevisto: "90"} (TROPPO SPAZIO IMPIEGATO)	Descrizione troppo lunga!	
5	TempoPrevisto < 0	TempoPrevisto < 0 [ERROR], tutti gli altri validi		{Titolo: "Allenamento del 09/06/2026", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.", TempoPrevisto: "-45"}	TempoPrevisto non valido!	

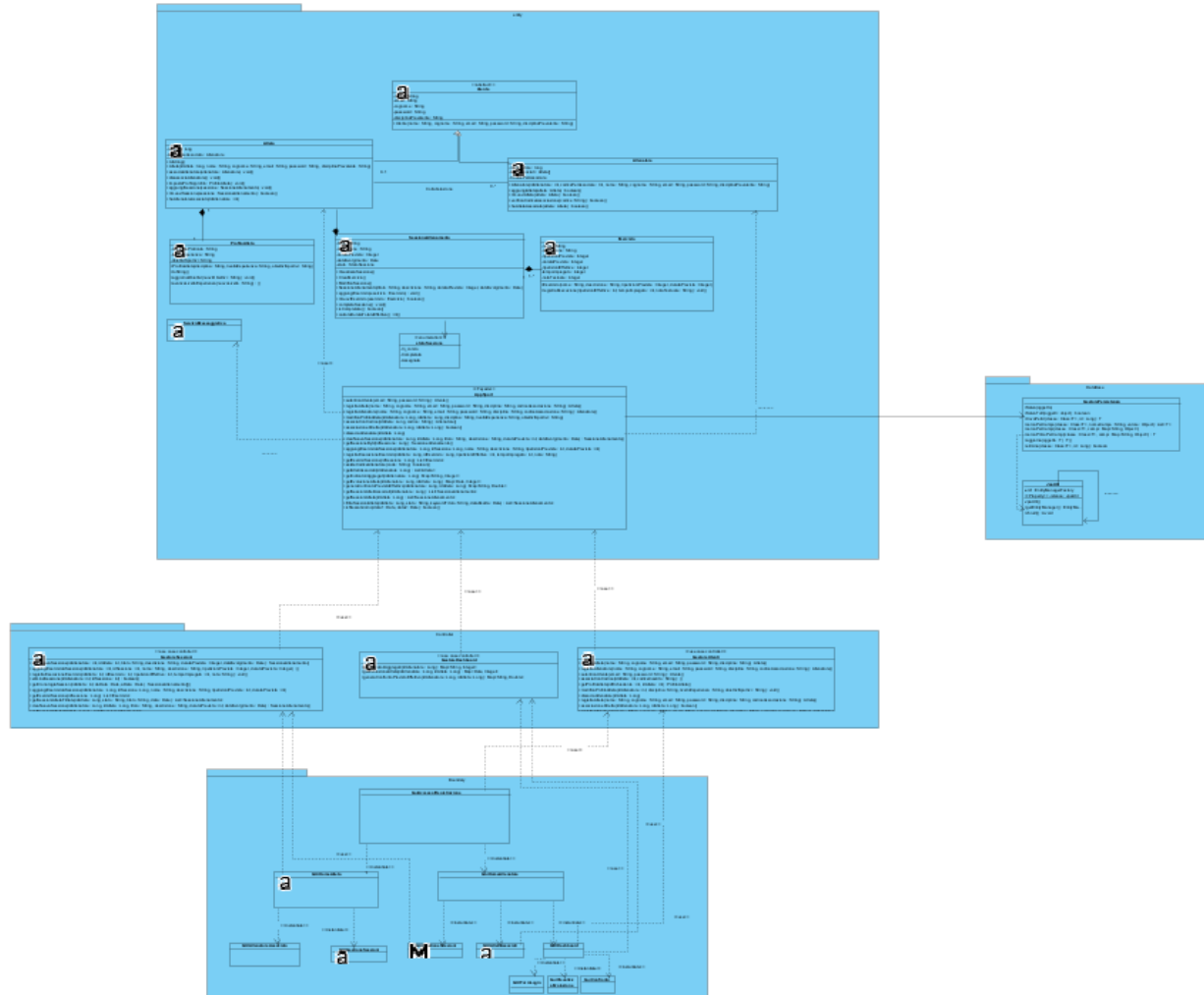


6	TempoPrevisto con simboli/lettere	TempoPrevisto con simboli/lettere [ERROR], tutti gli altri validi		{Titolo: "Allenamento del 09/06/2026", Descrizione: "Ogni esercizio va eseguito fino a cedimento con carichi bassi/medi.",, TempoPrevisto: "9u"}	Formato TempoPrevisto non valido!	
---	---	---	--	---	---	--



4. Progettazione

4.1 Diagramma delle classi



4.1.1 Traduzione classi ed associazioni

Traduzione classi ed associazioni:

- **Ereditarietà (Generalizzazione):** * La classe Utente è definita come **astratta** (<<abstract>>). Contiene gli attributi comuni a tutti gli utilizzatori del sistema (nome, email, cognome, password, disciplinaPrevalente).
 - Le classi **Atleta** e **Allenatore** estendono Utente, ereditandone gli attributi base e aggiungendo le proprie specificità (come idAtleta o idAllenatore).
- **Associazione "Collaborazione" (Atleta - Allenatore):**
 - Esiste una relazione diretta tra atleti e allenatori con cardinalità **0..1 a 0..***. Questo significa che un Allenatore può seguire una lista di AtletiAssociati (da 0 a infiniti), mentre un Atleta può avere al massimo un AllenatoreAssociato (oppure nessuno).



- **Composizioni:** La composizione indica un vincolo di esistenza forte ("ciclo di vita dipendente").
 - **Atleta e ProfiloAtleta (1 a 1):** Un profilo non può esistere senza l'atleta a cui appartiene. Se l'atleta viene eliminato dal sistema, anche il suo profilo scompare.
 - **Atleta e SessioneAllenamento (1 a 0..*):** Un atleta è proprietario delle sue sessioni. Una sessione specifica appartiene a un solo atleta.
 - **SessioneAllenamento ed Esercizio (1 a 1..*):** Una sessione funge da contenitore rigido per uno o più esercizi. Un esercizio specifico (inteso come istanza di esecuzione con ripetizioniEffettive, non come dizionario globale) esiste solo all'interno della sessione in cui è stato programmato.
- **ServizioMessaggistica come Interfaccia:** Non rappresenta un'entità interna che memorizza dati nel database del sistema, ma funge da **interfaccia di astrazione** tra il sistema e un servizio di messaggistica esterno (che si occuperà dell'invio delle notifiche *implementazione futura*)
- **Enumerazione e Dipendenze:**
 - La classe SessioneAllenamento ha un attributo di tipo StatoSessione (che è una <<enumeration>> con valori In_corso, Completata, Assegnata), garantendo un controllo rigoroso sul ciclo di vita della sessione stessa.

Scelte architetturali effettuate:

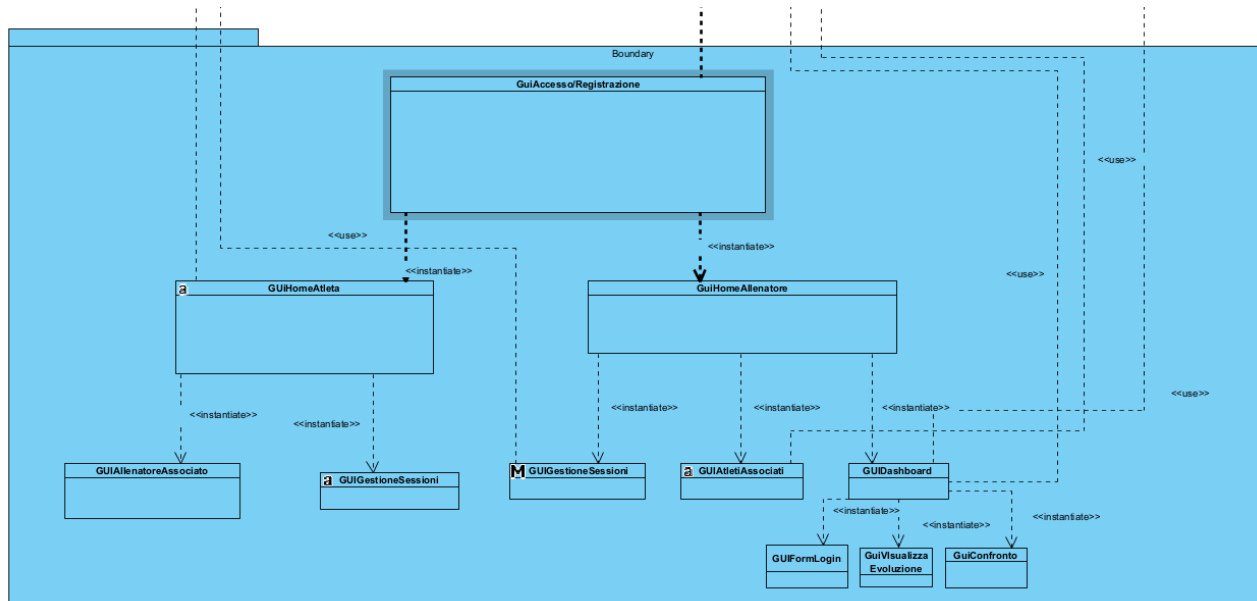
- **Pattern Facade**
 - L'architettura del sistema è stata rifinita introducendo la classe AppSport, che agisce come Facade (Facciata) per l'intero sistema. Questa classe rappresenta l'unica interfaccia unificata e il punto di accesso principale per la logica di business. Come evidenziato dalle relazioni di dipendenza (<<use>>) nel diagramma, il Facade è l'unico componente che ha il privilegio di interagire e comunicare direttamente con le classi del pacchetto entity. Il suo scopo è nascondere la complessità e le interazioni interne del modello di dominio, fornendo un set di metodi ad alto livello (es. creaNuovaSessione(), registraAtleta()) pronti per essere consumati dai livelli superiori.
- **Architettura di Controllo**
 - Al livello Control troviamo i gestori dei casi d'uso (GestoreSessioni, GestoreUtenti, GestoreDashboard). L'introduzione del Facade ridefinisce e snellisce il loro ruolo:
 - **Ricezione Input:** I Controller ricevono i dati e le interazioni provenienti dalle classi Boundary
 - **Delegazione:** I Controller non istanziano né manipolano mai direttamente le entità. Il loro compito è tradurre e instradare gli input delle Boundary in specifiche chiamate ai metodi offerti da AppSport. Questo meccanismo è rappresentato visivamente dalle frecce di dipendenza (<<use>>) che partono dai singoli Controller e puntano verso il Facade.

I vantaggi architetturali ottenuti da queste scelte favoriscono il disaccoppiamento e la coesione.

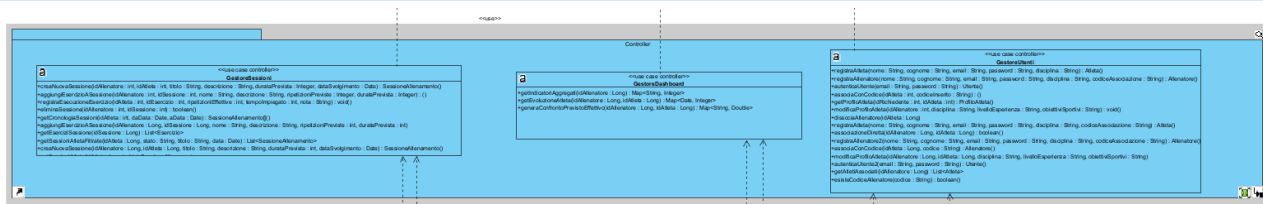
4.1.2 Pattern BCED



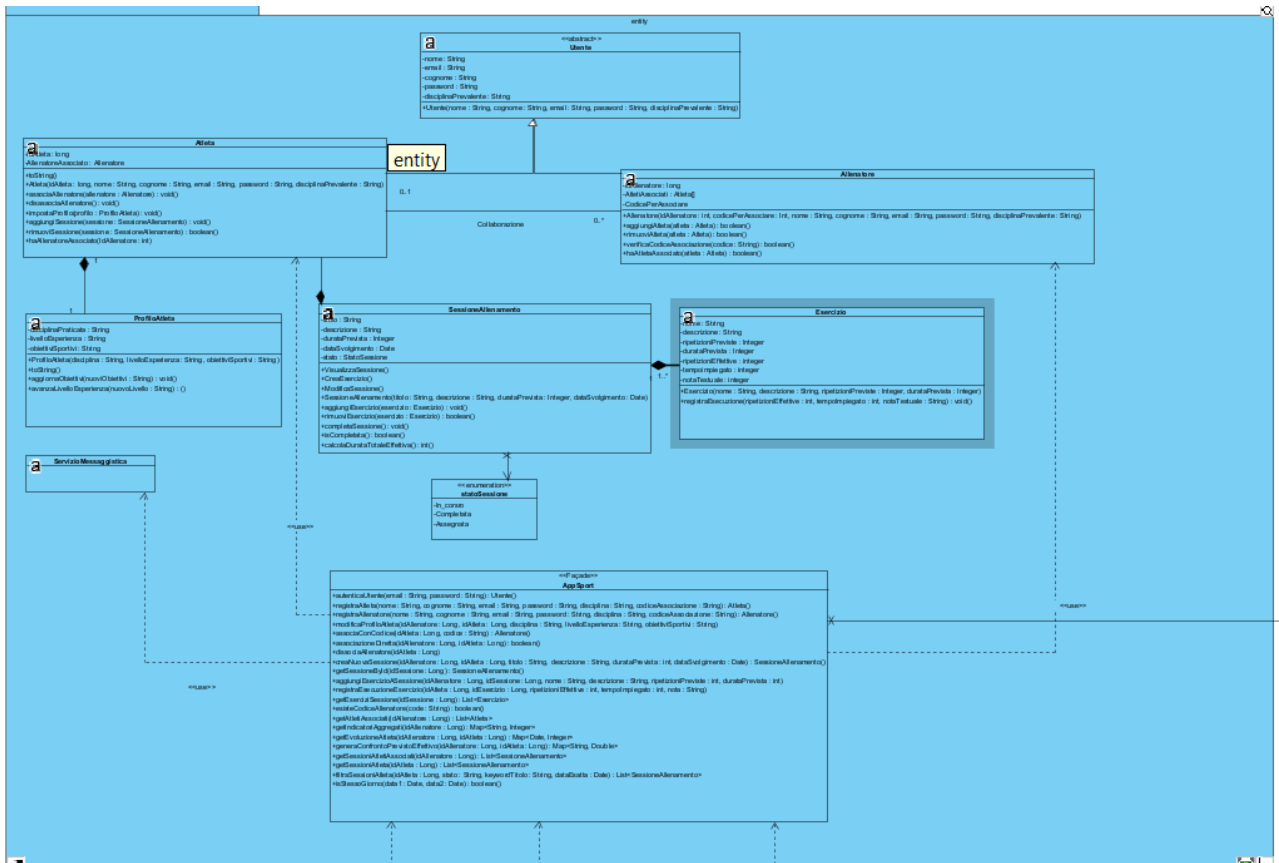
4.1.2.1 Package Boundary



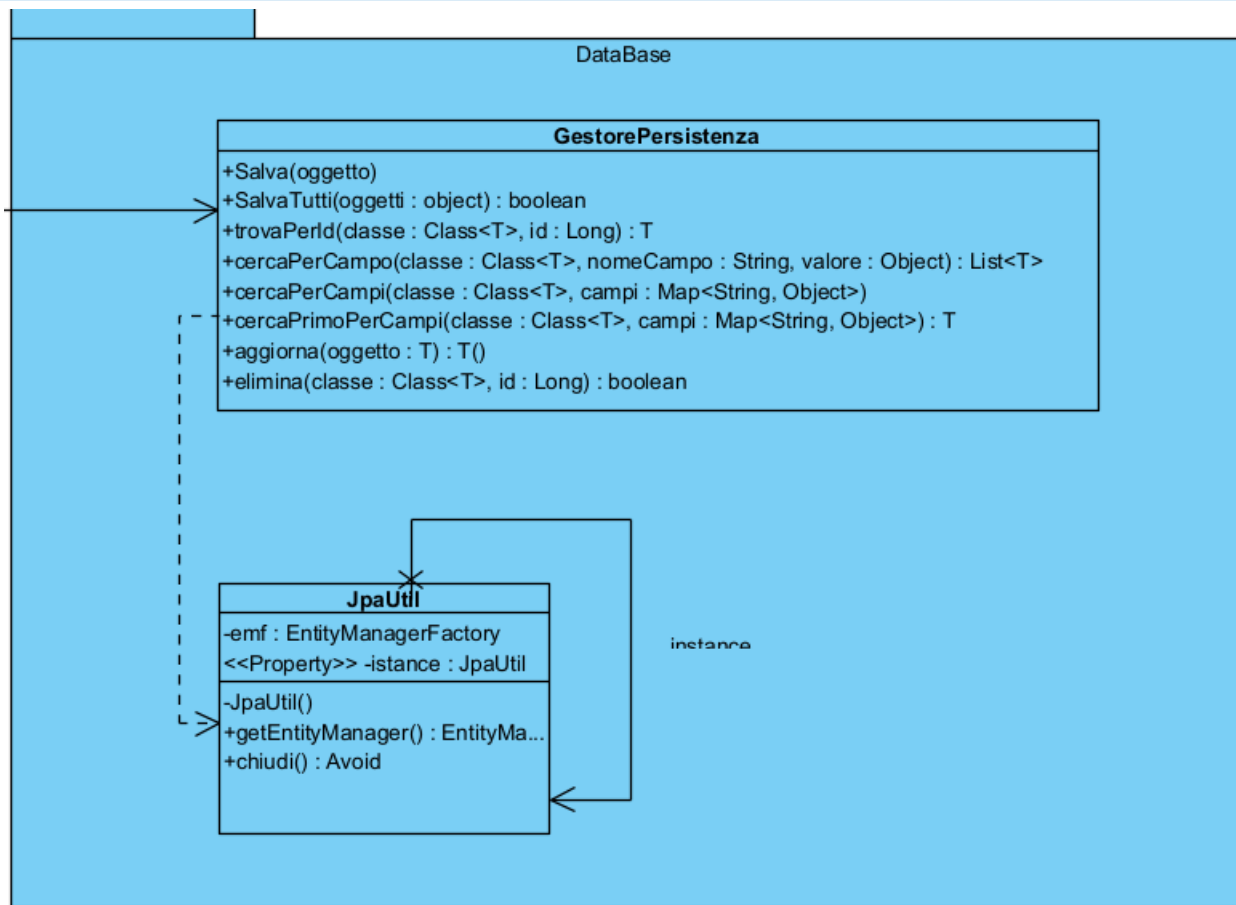
4.1.2.2 Package Controller



4.1.2.3 Package Entity



4.1.2.4 Package Database



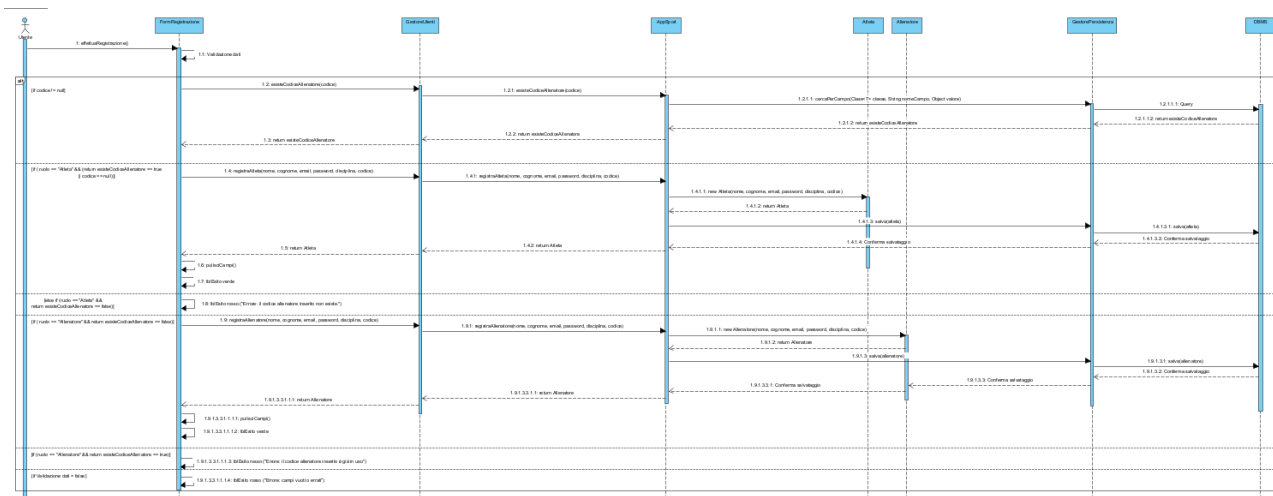


4.2 Diagrammi di sequenza

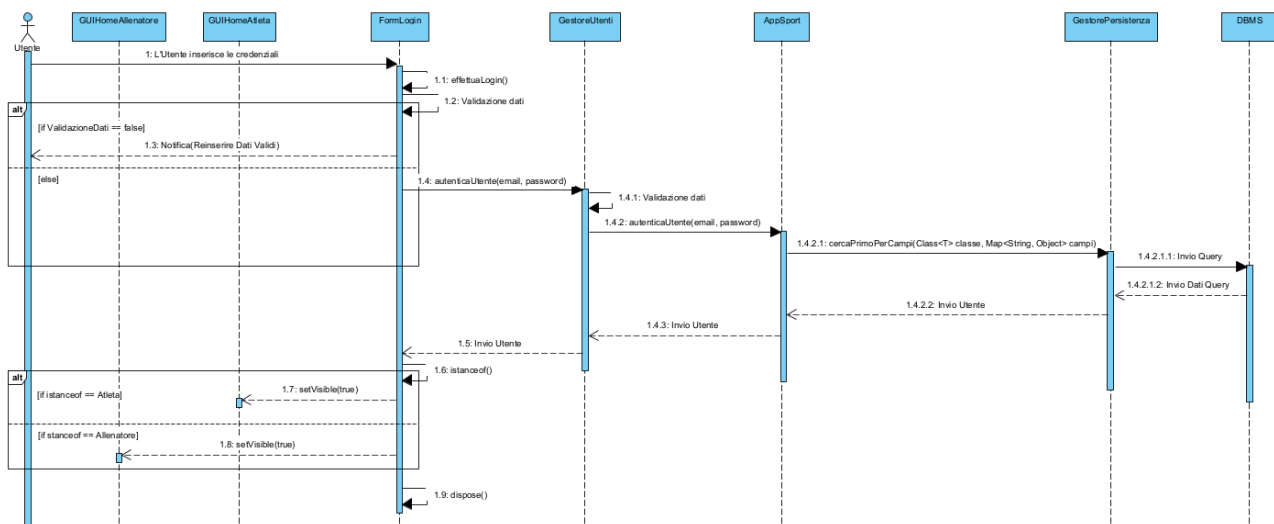


Si riportino di seguito i diagrammi di sequenza di progetto per il/i casi d'uso sviluppati fino alla codifica in Java.

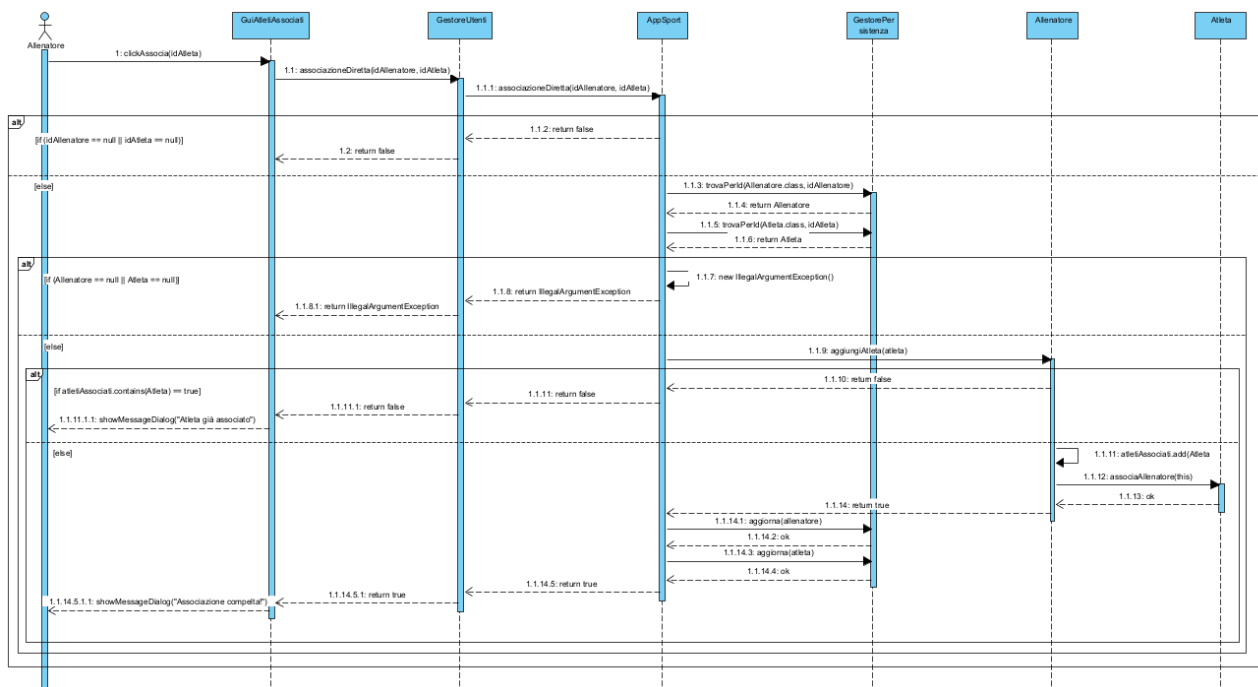
4.2.1 Registrazione



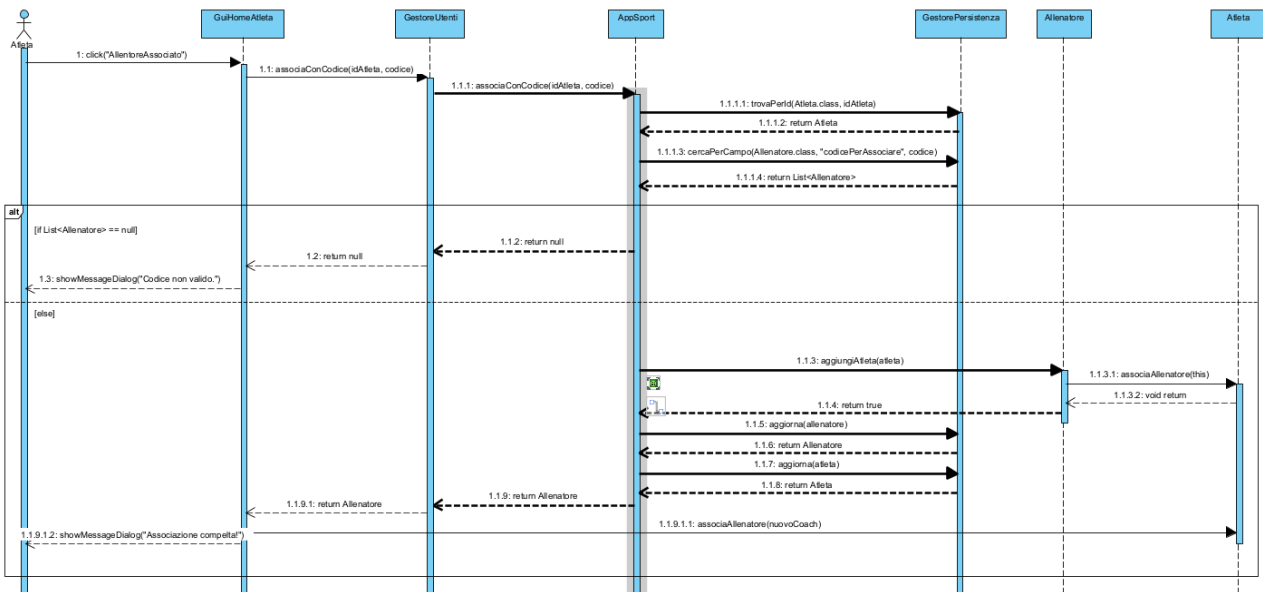
4.2.2 Accesso



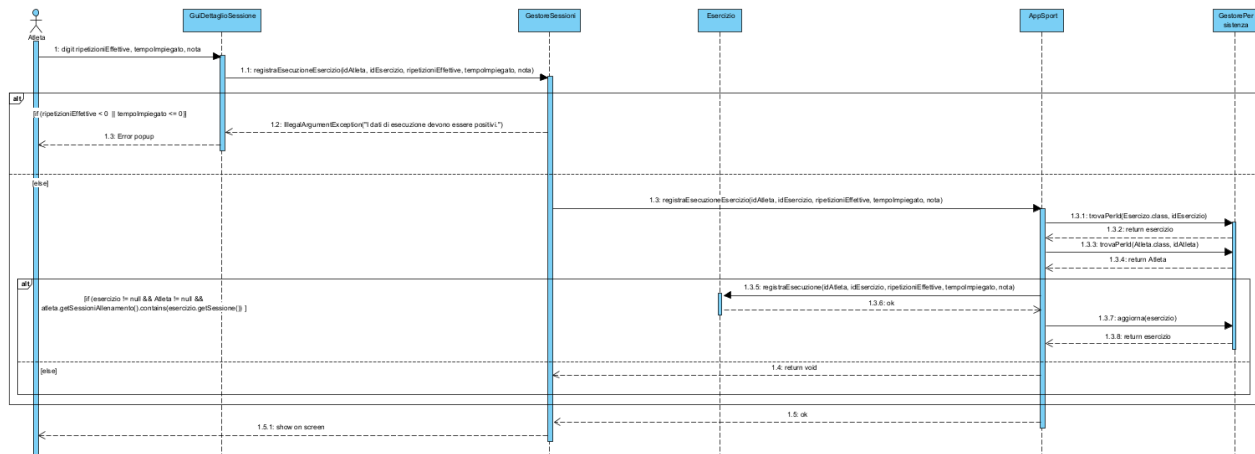
4.2.3 AssociazioneDiretta



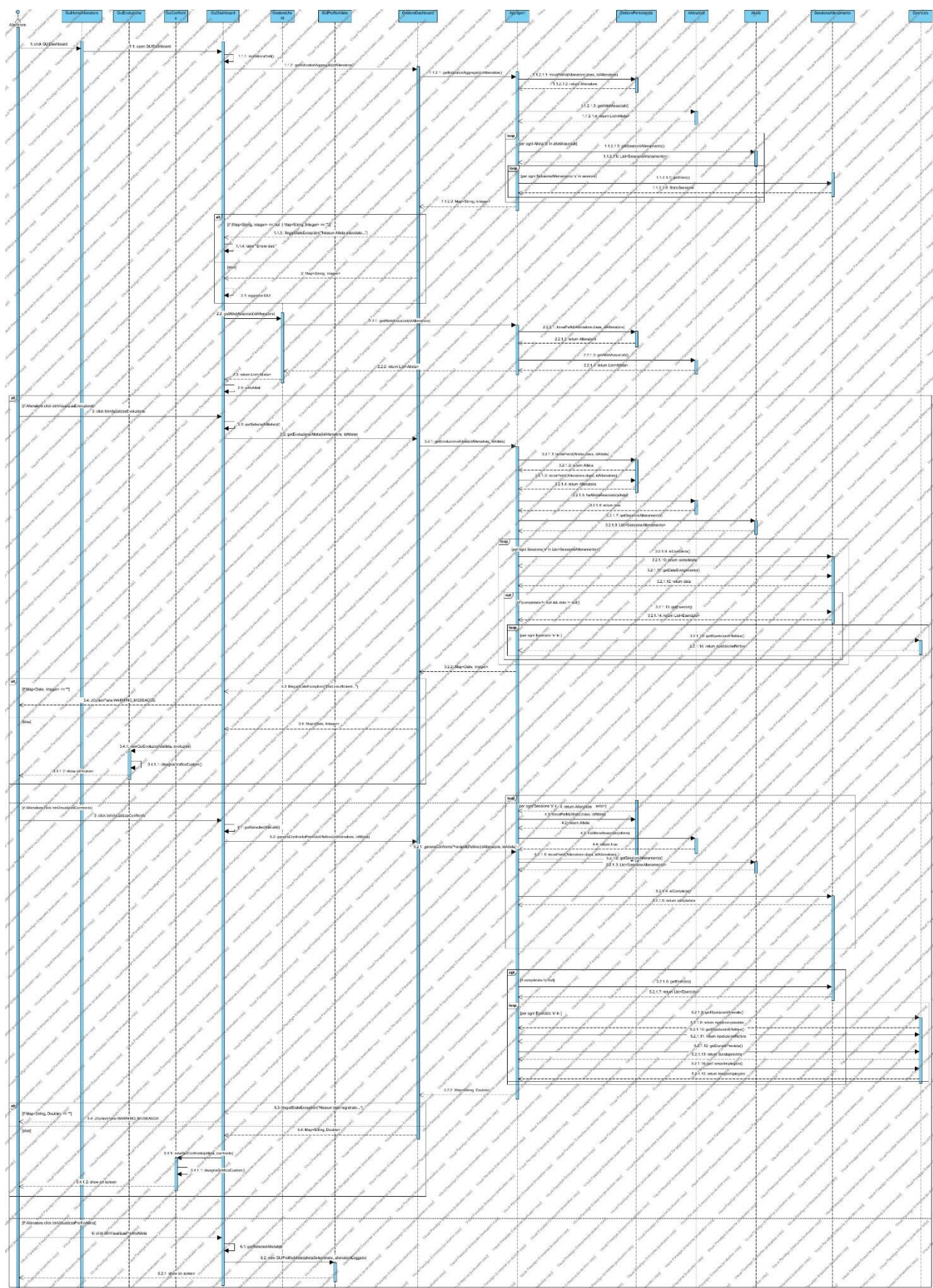
4.2.4 AssociazioneCodiceUnivoco



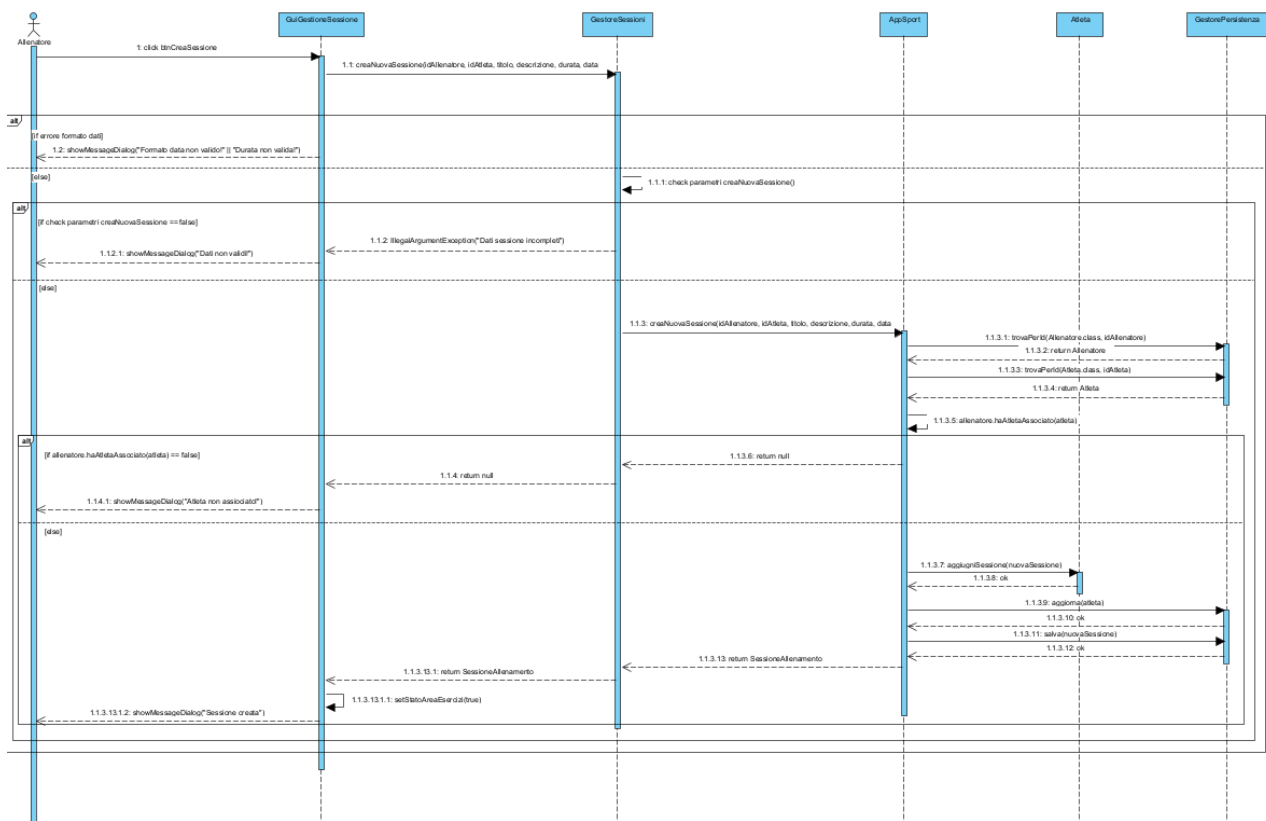
4.2.5 RegistrazioneRisultati



4.2.6 MonitoraggioDashboard



4.2.7 CreazioneSessione



I sequence progettati sono stati fondamentali per la corretta implementazione dell'applicazione software ed ha fatto nascere la necessità di definire ulteriori classi, metodi e funzioni, che hanno arricchito passo dopo passo il [Diagramma delle Classi di Progettazione](#), fino ad ottenere la seguente versione finale: *Non è nata alcuna necessità di apportare modifiche al diagramma*

Riportare il Diagramma delle Classi di Progettazione di Dettaglio

5. Implementazione

Non includere il codice sorgente, ma descrivere l'implementazione in Java, descrivendo gli artefatti di codifica:

- Elencare:
 - o package, classi, tipi di eccezione definiti
- Elencare gli artefatti necessari per l'installazione ed esecuzione del programma, senza ovviamente l'ambiente di sviluppo come Eclipse (DB h2, eventuali librerie e versioni di Java che l'utilizzatore deve avere installati, file .class, .jar, ...)
- Produrre un eventuale diagramma di deployment
- Eventualmente inserire la documentazione del codice prodotta con Javadoc (relativamente alle funzionalità implementate)



5.1 Package Database

Classi:

- **JpaUtil:** Classe di utilità fondamentale per la gestione del ciclo di vita di JPA (Java Persistence API). Implementa il Pattern Design Singleton per garantire che l'istanza del EntityManagerFactory sia unica all'interno dell'intera applicazione, ottimizzando l'uso delle risorse e le connessioni al database. Espone i metodi per ottenere l'istanza corrente, per creare un nuovo EntityManager (necessario per ogni transazione) e per chiudere in sicurezza il factory alla terminazione del programma.
- **GestorePersistenza:** Classe che funge da strato di astrazione e ordinamento per tutte le operazioni CRUD (Create, Read, Update, Delete). Utilizza metodi generici basati sui Java Generics per poter operare in modo polimorfico su qualsiasi classe del pacchetto entity (es. Atleta, SessioneAllenamento) senza duplicazione di codice. Gestisce programmaticamente l'apertura delle transazioni, i blocchi try-catch-finally per la sicurezza dei dati, l'esecuzione dei metodi di Hibernate (persist, merge, find, remove), l'esecuzione di query tipizzate

Tipi di Eccezione Definiti:

- Il pacchetto non definisce eccezioni, ma intercetta in modo centralizzato e strutturato le eccezioni standard fornite da jakarta.persistence (come PersistenceException, RollbackException o IllegalArgumentException) e le eccezioni generali di runtime (Exception), tracciandone lo stack trace per garantire la diagnostica senza interrompere bruscamente l'esecuzione dell'applicazione.

5.2 Package Entity

Classi:

- **Utente:** Rappresenta la base concettuale dell'intero sistema di autenticazione. Raccoglie tutti gli attributi anagrafici e le credenziali di accesso (come nome, cognome, email e password) condivisi da chiunque acceda alla piattaforma. Essendo astratta, non esiste come entità isolata, ma propaga la sua struttura informativa direttamente alle classi che la estendono.
- **Atleta e Allenatore:** Costituiscono le due tipologie di utenti effettivi del sistema. Estendono la classe base Utente ereditandone le credenziali e implementano comportamenti e associazioni specifici per il proprio ruolo. A livello strutturale, determinano la creazione delle rispettive tabelle all'interno del database relazionale.
- **ProfiloAtleta:** Rappresenta un insieme coeso di dati specialistici (disciplina praticata, livello di esperienza e obiettivi sportivi) che definiscono il background tecnico di un atleta. Dal punto di vista strutturale, non possiede un'identità autonoma o una propria tabella dedicata; le sue informazioni sono incorporate direttamente all'interno del record dell'atleta a cui appartiene, garantendo una forte coesione dei dati.
- **SessioneAllenamento ed Esercizio:** Sono le entità operative che tracciano la pianificazione e l'esecuzione dell'attività fisica. Sono legate da una relazione di composizione gerarchica (uno-a-molti): una sessione è composta da un elenco di



singoli esercizi. La logica di gestione è configurata in modo rigido in modo che il ciclo di vita degli esercizi dipenda totalmente dalla sessione: la cancellazione di un allenamento o la rimozione di un esercizio dalla lista comporta l'automatica eliminazione definitiva dei dati corrispondenti nel database.

- **StatoSessione**: Un tipo di dato strutturato che definisce gli stati ammissibili nel ciclo di vita di un allenamento (ASSEGNATA, IN_CORSO, COMPLETATA).
- **AppSport** (Facade): Rappresenta il fulcro e il punto di accesso unificato (Facade) all'intero modello del dominio. Il suo compito principale è incapsulare la complessità delle interazioni tra le varie entità e il sistema di persistenza dei dati. Esponendo macro-operazioni ad alto livello, permette ai Controller del livello superiore (Control) di eseguire flussi di business complessi senza che questi debbano conoscere i dettagli implementativi o le singole interrogazioni dirette al database.

Tipi di Eccezione Definiti:

- Non sono state create classi di eccezione custom. La logica di validazione del dominio si affida al lancio di eccezioni standard Java, in particolare `IllegalArgumentException`, utilizzata nella classe `AppSport` per segnalare violazioni delle regole di business (es. atleta non trovato, autorizzazione negata durante la modifica del profilo).

5.3 Package Controller

Classi:

- **GestoreUtenti**: Coordina i casi d'uso legati all'accesso e alla gestione anagrafica. Riceve i dati utente dalla Boundary, si occupa di validare le credenziali di login e verifica la robustezza dei parametri in fase di registrazione. Superati i controlli formali, delega al Facade le operazioni di salvataggio e le logiche di associazione/dissociazione tra Allenatori e Atleti.
- **GestoreSessioni**: Gestisce il flusso informativo relativo agli allenamenti. Riceve in input i dati per la schedulazione e ne valida la coerenza temporale e logica (es. garantendo che le durate siano positive e i titoli non vuoti). Una volta confermata la validità, delega al Facade la creazione effettiva delle nuove sessioni, l'aggiunta di esercizi o la registrazione dei risultati effettivi. Funge inoltre da intermediario per delegare il recupero e il filtraggio delle sessioni.
- **GestoreDashboard**: Si occupa di orchestrare i dati necessari per la reportistica visiva. Riceve dalla vista gli input di selezione (es. l'ID dell'atleta da analizzare) e richiama le macro-operazioni dal Facade delegando a quest'ultimo i calcoli complessi per generare indicatori aggregati, mappe di evoluzione e scostamenti tra il lavoro previsto e quello effettivo, confezionando le mappe dati da restituire alla GUI.

Tipi di Eccezione Definiti:

- Non sono state create eccezioni custom, ma si fa un uso delle eccezioni standard Java per gestire i flussi alternativi e di errore:



- **IllegalArgumentException:** Lanciata sistematicamente all'inizio di ogni metodo se i parametri di input provenienti dalla Boundary risultano mancanti (null), vuoti o non validi (es. valori negativi per tempo e ripetizioni).
- **IllegalStateException:** Utilizzata nel GestoreDashboard per intercettare i casi in cui i dati richiesti per generare i grafici sono logicamente assenti (es. un allenatore senza atleti associati o un atleta senza storico), bloccando l'operazione prima di inviare dati inconsistenti alla GUI.

5.4 Package Boundary

Classi:

- **GuiAccessoRegistrazione:** Il punto di ingresso (Entry Point) dell'applicazione, contenente il metodo main. Istanza l'architettura di base (Facade e Controller) e permette all'utente di scegliere se accedere o registrarsi.
- **FormLogin e FormRegistrazione:** Gestiscono i moduli di autenticazione e iscrizione al sistema. Adattano dinamicamente l'interfaccia in base alle scelte dell'utente (es. in fase di registrazione, il campo "Codice Associazione" cambia comportamento e obbligatorietà a seconda se si sceglie il ruolo Atleta o Allenatore).
- **GuiHomeAtleta:** La dashboard principale per l'atleta. Permette di visualizzare le sessioni assegnate filtrate per data, titolo o stato. Gestisce inoltre le logiche di interfaccia per l'associazione/dissociazione con il proprio allenatore, comunicando con il controller per l'effettivo salvataggio.
- **GuiHomeAllenatore:** Il menu principale per il Coach. Funge da snodo di navigazione verso le schermate di gestione sessioni, dashboard analitica e gestione del roster atleti (che andranno a completare il package).
- **GuiDettaglioSessione:** Schermata operativa dedicata all'atleta per compilare il feedback post-allenamento. Sblocca i campi di testo (Ripetizioni e Tempo impiegato) solo quando viene selezionato un esercizio specifico dalla JTable.
- **GuiProfiloAtleta:** Un'interfaccia ibrida e dinamica. Se viene aperta da un Atleta, i campi sono di sola lettura. Se viene aperta da un Allenatore, i campi si sbloccano permettendo la modifica della disciplina, del livello e degli obiettivi, delegando poi il salvataggio al GestoreUtenti.
- **GuiDettaglioSessione:** Schermata operativa dedicata all'Atleta per l'analisi di una specifica sessione di allenamento. Permette di consultare l'elenco degli esercizi e di registrarne i risultati effettivi (ripetizioni e tempi). Implementa un vincolo logico lato client abilitando i campi di inserimento e il pulsante di salvataggio esclusivamente dopo la selezione di una riga dalla tabella degli esercizi.
- **GuiGestioneSessioni:** Schermata operativa a disposizione dell'Allenatore per la creazione e la strutturazione dei piani di allenamento. Guida il flusso di lavoro del coach vincolando in modo rigido l'aggiunta dei singoli esercizi alla preventiva e corretta creazione della sessione madre nel sistema, attraverso una gestione dinamica dello stato abilitato/disabilitato dei campi di testo.
- **GuiAtletiAssociati:** Interfaccia di gestione del roster o "scuderia" dell'Allenatore. Mostra in una griglia strutturata l'elenco degli atleti attualmente seguiti ed espone i comandi per effettuare una nuova associazione diretta digitando l'identificativo (ID) dell'atleta o per interrompere il legame (dissociazione) previa richiesta esplicita di conferma visiva tramite pop-up.



- **GuiDashboard:** Pannello di monitoraggio analitico e riassuntivo riservato all'Allenatore. Mostra a colpo d'occhio gli indicatori globali aggregati (totale atleti, sessioni assegnate, completate e in corso) ed espone un menu a tendina per selezionare un singolo atleta e navigare verso i dettagli relativi alle sue prestazioni individuali.
- **GuiEvoluzione:** Finestra di reportistica grafica avanzata per il tracciamento delle performance storiche. Estende i componenti standard di Java Swing effettuando l'override del metodo **paintComponent(Graphics g)** per calcolare le scale geometriche e disegnare da zero un grafico cartesiano personalizzato che illustra l'evoluzione temporale del volume di allenamento dell'atleta.
- **GuiConfronto:** Schermata di reportistica analitica complementare che si occupa di visualizzare il tasso di completamento delle sessioni. Mostra all'interno di una tabella di sola lettura il calcolo degli scostamenti percentuali tra i target ideali stabiliti dall'allenatore (ripetizioni e tempi previsti) e i risultati reali inseriti dall'atleta.
- **GuiProfiloAtleta:** Interfaccia ibrida e polimorfica il cui stato e i cui permessi variano dinamicamente in base al contesto di esecuzione. Se aperta da un Atleta per consultare la propria scheda, mostra i dati personali in modalità di sola lettura; se invece viene aperta dall'Allenatore responsabile, i campi di testo si abilitano permettendo la modifica della disciplina, del livello di esperienza e degli obiettivi sportivi dell'atleta selezionato.

Tipi di Eccezione Definiti:

- Non sono presenti eccezioni custom, ma si fa largo uso di blocchi try-catch all'interno dei listener (es. ActionListener) per intercettare le eccezioni sollevate dal livello Control (come IllegalArgumentException) e trasformarle in messaggi visivi tramite JOptionPane.showMessageDialog o etichette di errore (JLabel rosse), fornendo un feedback immediato all'utente senza far crashare il programma.

5.5 Diagramma di Deployment

Ambiente di Esecuzione (Prerequisiti di Sistema)

- JRE / JDK: Versione Java 24.
- Database: Server MySQL attivo.

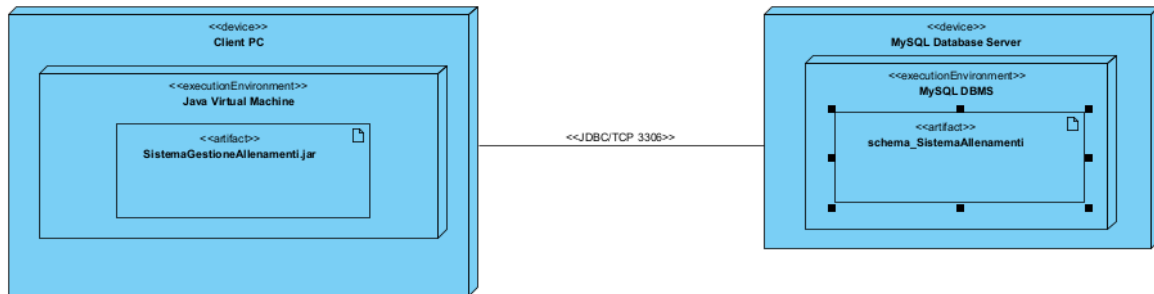
2. Librerie (Dipendenze)

- JPA/Hibernate: hibernate-core (v. 6.4.4.Final)
- Driver Database: mysql-connector-j (v. 8.3.0)
- Libreria GUI: forms_rt (v. 7.0.3)

3. File Applicativi e di Configurazione

- Eseguitibile: File .jar dell'applicazione.
- Configurazione Database: File persistence.xml





6. Testing

6.1 Test Strutturale

6.1.1 Complessità ciclomatica

Si intende costruire il Control Flow Graph per due dei metodi delle classi implementate e mostrare il calcolo del numero ciclomatico e i percorsi linearmente indipendenti.

6.1.1.1 creaNuovaSessione – AppSport

```
public SessioneAllenamento creaNuovaSessione(Long idAllenatore,
Long idAtleta, String titolo, String descrizione, int durataPrevista,
Date dataSvolgimento) {
    Allenatore allenatore = db.trovaPerId(Allenatore.class,
idAllenatore); (1)
    Atleta atleta = db.trovaPerId(Atleta.class, idAtleta); (2)

    if (allenatore != null && atleta != null &&
allenatore.haAtletaAssociato(atleta)) (3) {

        // FIX DATA: Convertiamo java.util.Date in java.sql.Date
        java.sql.Date dataSQL = new
java.sql.Date(dataSvolgimento.getTime()); (4)

        SessioneAllenamento nuovaSessione = new
SessioneAllenamento(titolo, descrizione, durataPrevista, dataSQL);
(5)

        nuovaSessione.setAtleta(atleta); (6)
        atleta.aggiungiSessione(nuovaSessione); (7)

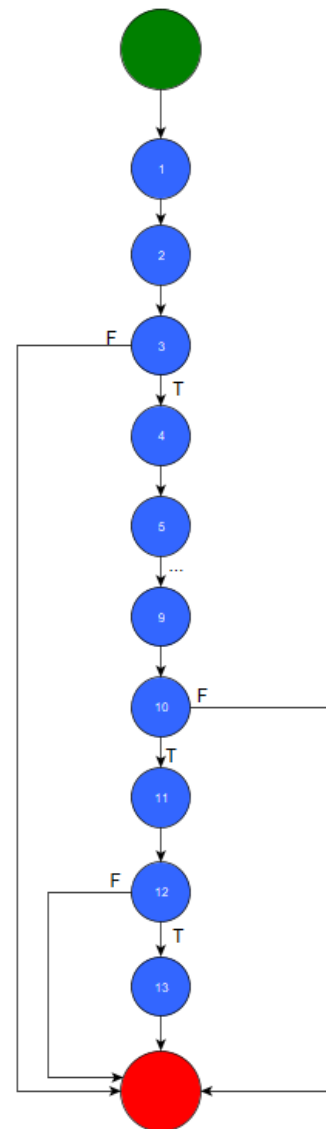
        db.aggiorna(atleta); (8)

        List<SessioneAllenamento> sessioni =
atleta.getSessioniAllenamento(); (9)
        if (sessioni != null && !sessioni.isEmpty()) (10) {
            SessioneAllenamento sessioneAggiornata =
sessioni.get(sessioni.size() - 1); (11)

            if (sessioneAggiornata.getIdSessione() == null) (12) {
                db.salva(nuovaSessione); (13)
                return nuovaSessione;
            }

            return sessioneAggiornata;
        }

        return nuovaSessione;
    }
    return null;
}
```



NUMERO CICLOMATICO:

- numero di regioni chiuse del grafo + 1 = 4
- numero di nodi predicati (3, 10, 12) + 1 = 4
- # archi - # nodi + 2 = (17 - 15) + 2 = 4



CAMMINI:

- 1) 1-2-3
- 2) 1-2-3-4-5-6-7-8-9-10
- 3) 1-2-3-4-5-6-7-8-9-10-11-12
- 4) 1-2-3-4-5-6-7-8-9-10-11-12-13

CASI di TEST

TC ID	Cammino Coperto	Descrizione	Condizioni Logiche	Input	Precondizioni	Output atteso	Output effettivo	Esito (PASS/FAIL)
TC_1	Green-1-2-3-Red	Allenatore o atleta non trovati, oppure non associati.	La condizione al punto 3 del CFG è false	idAllenatore=99, idAtleta=50, titolo="Corsa", durataPrevista=60, dataSvolgimento=2026-06-18	L'allenatore o l'atleta non esistono nel DB, oppure haAtletaAssociato restituisce false.	null	...	PASS/FAIL
TC_2	Green-1-2-3-4-5-6-7-8-9-10-Red	Allenatore e atleta validi, ma l'atleta non ha alcuna sessione registrata.	La condizione al punto 3 è true. La condizione al punto 10 è false	idAllenatore=1, idAtleta=2, titolo="Corsa", durataPrevista=60, dataSvolgimento=2026-06-18	Allenatore e atleta esistono e sono associati. getSessioniAllenamento() restituisce null o lista vuota.	Oggetto nuovaSessione (non salvato esplicitamente via db.salva()).	...	PASS/FAIL
TC_3	Green-1-2-3-4-5-6-7-8-9-10-11-12-Red	L'atleta ha sessioni e l'ultima inserita ha già un ID valido nel database.	La condizione al punto 3 è true. La condizione al punto 10 è true. La condizione al punto 12 è false	idAllenatore=1, idAtleta=2, titolo="Corsa", durataPrevista=60, dataSvolgimento=2026-06-18	Allenatore e atleta associati. L'atleta ha sessioni e l'ultima ha idSessione != null.	Oggetto sessioneAggiornata (l'ultima della lista).	...	PASS/FAIL
TC_4	Green-1-2-3-4-5-6-7-8-9-10-11-12-13-Red	L'atleta ha sessioni, ma l'ultima inserita non ha un ID, quindi viene salvata la nuova.	La condizione al punto 3 è true. La condizione al punto 10 è true. La condizione al punto 12 è true	idAllenatore=1, idAtleta=2, titolo="Corsa", durataPrevista=60, dataSvolgimento=2026-06-18	Allenatore e atleta associati. L'atleta ha sessioni e l'ultima ha idSessione == null.	Oggetto nuovaSessione (salvato nel DB tramite db.salva).	...	PASS/FAIL

6.1.1.2 associaConCodice - AppSport



```

public Allenatore associaConCodice(Long idAtleta, String codice) {
    if (codice == null || codice.trim().isEmpty()) (1) {
        return null;
    }

    Atleta atleta = db.trovaPerId(Atleta.class, idAtleta) (2);
    if (atleta == null) (3) {
        throw new IllegalArgumentException("Atleta non trovato nel
sistema.") (4);
    }

    List<Allenatore> allenatori = db.cercaPerCampo(Allenatore.class,
"codicePerAssociare", codice.trim()) (5);

    if (allenatori != null && !allenatori.isEmpty())(6) {
        Allenatore allenatore = allenatori.get(0); (7)

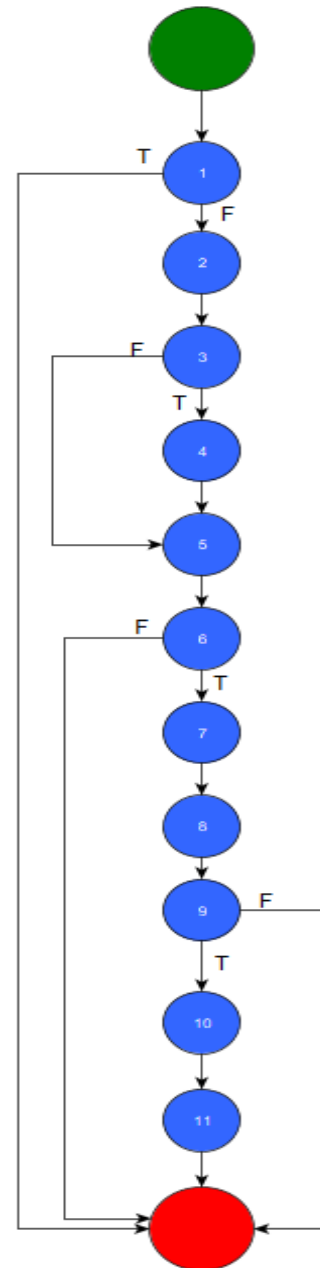
        boolean associato = allenatore.aggiungiAtleta(atleta); (8)

        if (associato) (9) {
            db.aggiorna(allenatore); (10)
            db.aggiorna(atleta); (11)
        }

        return allenatore;
    }

    return null;
}

```



NUMERO CICLOMATICO:

- numero di regioni chiuse del grafo + 1 = 5
- numero di nodi predicati (1, 3, 6, 9) + 1 = 5
- # archi - # nodi + 2 = (16 - 13) + 2 = 5

CAMMINI:

- 1) 1
- 2) 1-2-3-4-5-6
- 3) 1-2-3-4-5-6-7-8-9
- 4) 1-2-3-4-5-6-7-8-9-10-11

CASI di TEST

TC ID	Cammino Coperto	Descrizione	Condizioni Logiche	Input	Precondizioni	Output atteso	Output effettivo	Esito (PASS/FAIL)
-------	-----------------	-------------	--------------------	-------	---------------	---------------	------------------	-------------------

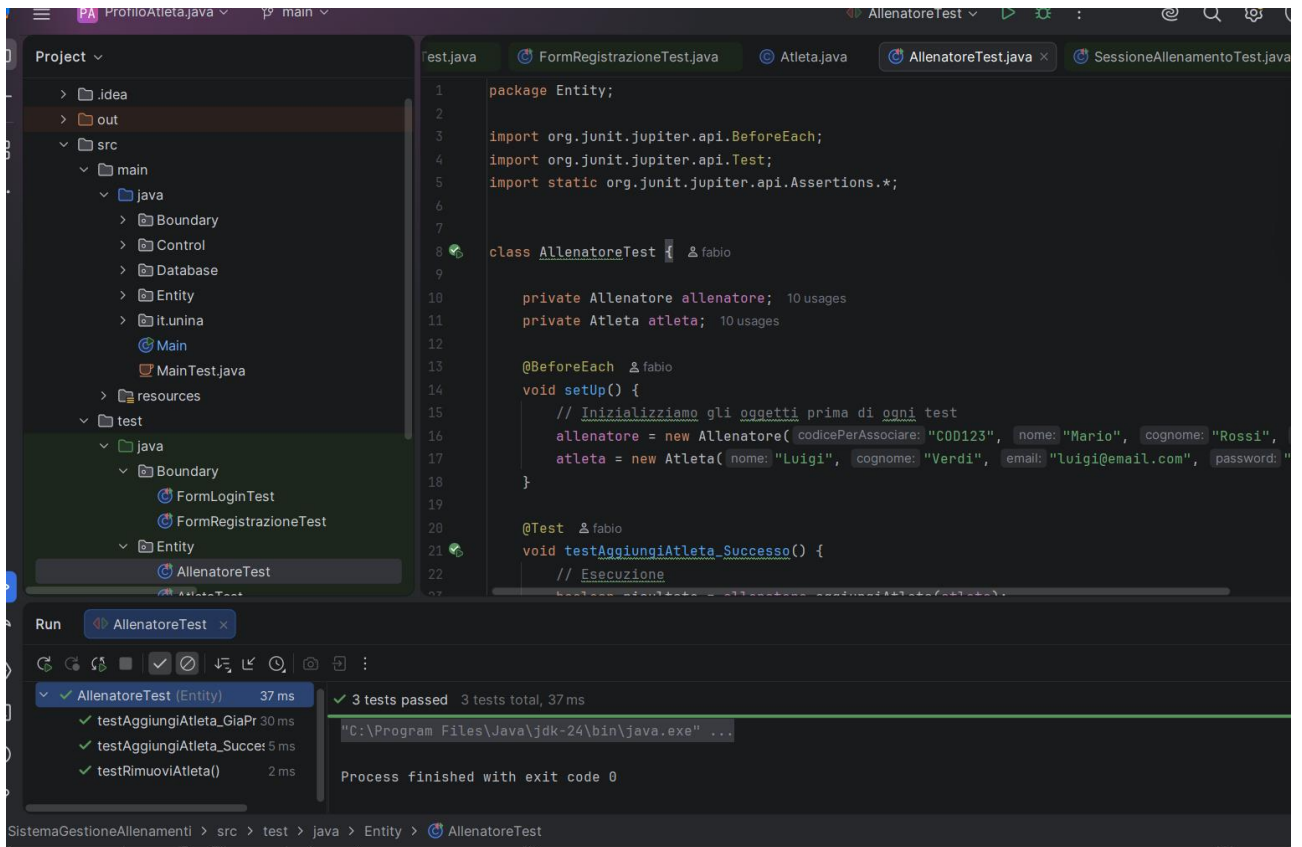


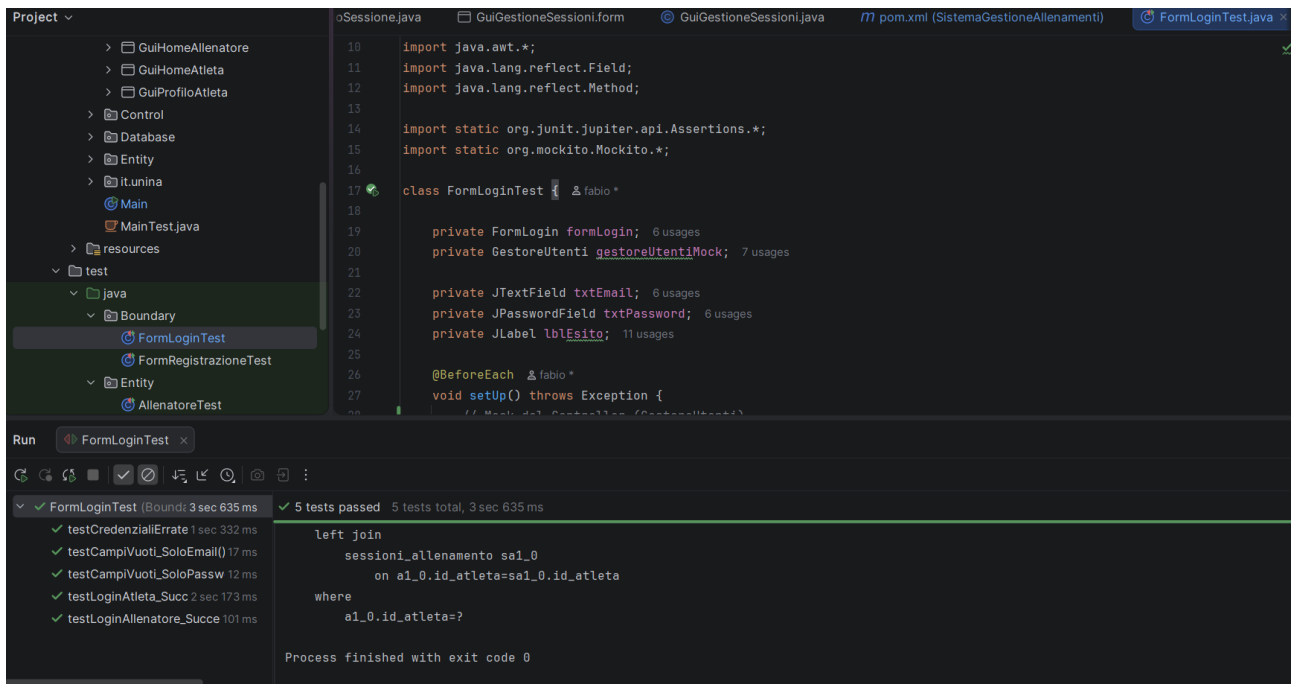
TC_1	Inizio - 1 - Fine	Codice di associazione nullo o vuoto.	Condizione 1: True	idAtleta = 10L codice = ""	Nessuna.	null	...	PASS/FAIL
TC_2	Inizio - 1 - 2 - 3 - 4 - 5 - 6 - Fine	Atleta non trovato nel sistema.	Condizione 1: False Condizione 3: True Condizione 6: False	idAtleta = 999L codice = "ALL_01"	L'atleta con ID 999 non è presente nel database. Nessun allenatore ha codice "ALL_01".	IllegalArgumentException("Atleta non trovato nel sistema.")	...	PASS/FAIL
TC_3	Inizio - 1 - 2 - 3 - 5 - 6 - Fine	Atleta esistente, ma nessun allenatore associato al codice inserito.	Condizione 1: False Condizione 3: False Condizione 6: False	idAtleta = 10L codice = "ERR_COD"	L'atleta con ID 10 esiste nel sistema. Non esiste alcun allenatore con codice "ERR_COD".	null	...	PASS/FAIL
TC_4	Inizio - 1 - 2 - 3 - 5 - 6 - 7 - 8 - 9 - Fine	Atleta e allenatore trovati, ma l'aggiunta fallisce internamente.	Condizione 1: False Condizione 3: False Condizione 6: True Condizione 9: False	idAtleta = 10L codice = "ALL_01"	L'atleta con ID 10 esiste. L'allenatore con codice "ALL_01" esiste, ma il metodo aggiungiAtleta() restituisce false.	Ritorna l'oggetto allenatore (senza aggiornare il DB).	...	PASS/FAIL
TC_5	Inizio - 1 - 2 - 3 - 5 - 6 - 7 - 8 - 9 - 10 - 11 - Fine	Associazione completata con successo e persistita nel DB.	Condizione 1: False Condizione 3: False Condizione 6: True Condizione 9: True	idAtleta = 10L codice = "ALL_01"	L'atleta con ID 10 esiste. L'allenatore con codice "ALL_01" esiste e il metodo aggiungiAtleta() restituisce true.	Ritorna l'oggetto allenatore aggiornato.	...	PASS/FAIL

6.2 JUnit – Test di Unità

Riportare a scopo esemplificativo alcuni casi di utilizzo di **JUnit** per testare alcune classi del progetto, il framework di testing di unità automatizzato per il linguaggio di programmazione Java.







6.3 Test funzionale

Segue una descrizione in forma tabellare dei risultati dell'esecuzione dei test funzionali precedentemente pianificati.

AllenatoreTest									
TEST SUITE									
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese	Output ottenuti	Post-condizioni ottenute	Esito (FAIL, PASS)
1	Aggiungi atleta con successo	Atleta non ancora associato all'allenatore [VALID]	Oggetti Allenatore e Atleta inizializzati correttamente	atleta (Luigi Verdi)	true	L'atleta viene inserito nella lista dell'allenatore; l'atleta aggiorna il suo riferimento all'allenatore associato	true	L'atleta è presente nella lista dell'allenatore e l'atleta punta a allenatore	PASS
2	Aggiungi atleta già presente	Atleta già presente nella lista dell'allenatore [ERROR]	L'atleta è già stato precedentemente aggiunto all'allenatore	atleta (Luigi Verdi)	false	Nessun duplicato inserito; la dimensione della lista degli atleti associati rimane pari a 1	false	La lista non contiene duplicati (dimensione della lista immutata)	PASS
3	Rimuovi atleta associato	Rimozione di un atleta regolarmente associato [VALID]	L'atleta è presente nella lista dell'allenatore prima della rimozione	atleta (Luigi Verdi)	true	L'atleta viene rimosso dalla lista dell'allenatore; il riferimento all'allenatore nell'oggetto atleta diventa null	true	L'atleta non è più nella lista e il suo riferimento all'allenatore è disassociato (null)	PASS



TEST SUITE

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese	Output ottenuti	Post-condizioni ottenute	Esito (FAIL, PASS)
1	Tutti input validi	Email valida, Password valida	Utente registrato nel sistema come Atleta	{txtEmail: "mario.rossi@email.it", txtPassword: "Password1!"}	Login Atleta riuscito!	Apertura GuiHomeAtleta e chiusura della form di login	Login Atleta riuscito!	Il Cliente ha effettuato il login e ha ottenuto l'accesso alla dashboard Atleta	PASS
2	Password troppo breve	Email valida, Password < 6 caratteri [ERROR]	Nessuna	{txtEmail: "mario.rossi@email.it", txtPassword: "Pass1"}	Messaggio di errore credenziali o validazione formale	Nessuna modifica dello stato	Errore: Credenziali errate. (o errore di validazione)	Form di login bloccata in attesa di input validi	PASS
3	Password vuota	Email valida, Password vuota [ERROR]	Nessuna	{txtEmail: "mario.rossi@email.it", txtPassword: ""}	Errore: compila tutti i campi.	Intercettazione Boundary (Nessuna interazione con il Controller)	Errore: compila tutti i campi.	Form di login bloccata in attesa di input validi	PASS
4	Email senza chiocciola	Email senza '@' [ERROR], Password valida	Nessuna	{txtEmail: "mario.rossiemail.it", txtPassword: "Password1!"}	Messaggio di errore formato email o credenziali errate	Nessuna modifica dello stato	Errore: Credenziali errate. (o errore di formato)	Form di login bloccata in attesa di input validi	PASS
5	Email vuota	Email vuota [ERROR], Password valida	Nessuna	{txtEmail: "", txtPassword: "Password1!"}	Errore: compila tutti i campi.	Intercettazione Boundary (Nessuna interazione con il Controller)	Errore: compila tutti i campi.	Form di login bloccata in attesa di input validi	PASS

